JIT-Agent

通过即时（*JIT*）支架演化扩展支架智能

LV-NUS Lab

摘要

智能体能力并非仅由模型决定。智能体支架（agent harness）涵盖记忆管理、规划策略、行动协议以及工具/技能编排，它对系统能力的贡献可能超过底层基础模型。然而，支架设计仍依赖人工、面向特定任务，且从根本上难以扩展。我们提出 JIT-Agent：一个经过训练、能够为任意现成智能体式大语言模型即时合成任务自适应智能体支架的支架智能模型。我们将智能体支架形式化为一种由固定四模块协议约束、可组合且可由机器生成的制品，并训练 JIT-Agent 针对当前任务定制支架、为稳定可靠的执行修复支架，以及通过从不断扩展的既有支架配置档案中提炼性能信号来自我演化。以 JIT-Agent 作为支架助手后，DeepSeek-V4-Flash 在 DeepSearchQA（+9.1）和 OdysseyBench（+4.3）上超过 GPT-5.6，而本已强大的 GLM-5.2 最多提升 +20.2 分。在受控评测中，JIT-Agent 生成的支架在性能上可与 OpenCode、Claude Code 等成熟智能体运行时竞争，并持续提升 DeepSeek V4、Mimo-V2.5 和 Qwen3.6 等不同规模的模型家族。据我们所知，JIT-Agent 是首个专为即时支架生成而设计的模型，由此确立了支架智能这一与模型规模扩展正交、可训练、可迁移且能持续积累的智能体能力维度。

网站 https://bingreeky.github.io/JIT-site
GitHub https://github.com/bingreeky/JIT
Hugging Face https://huggingface.co/JIT-Agent

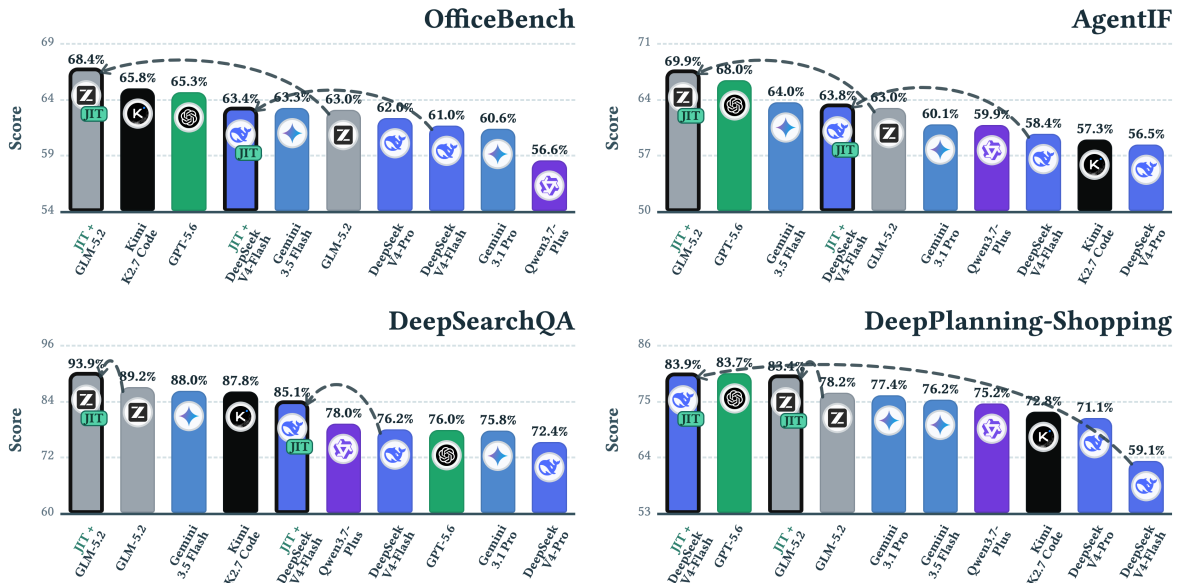


图 1: 四个代表性智能体基准上的排行榜。 JIT-Agent 生成的支架在深度研究、日常工作、规划和工作区任务上持续提升底层骨干智能体，表明即时支架合成带来的收益超越了单纯的模型规模扩展。

1 引言

智能体支架至关重要。大语言模型智能体的能力由两个紧密耦合的因素共同决定：产生推理与行动的基础模型，以及将该模型置于闭环执行环境中的智能体支架 (Meng 等, 2026b; Zhou 等, 2026; Ning 等, 2026)。支架决定保留哪些历史、如何形成局部意图、开放哪些工具或技能、如何执行行

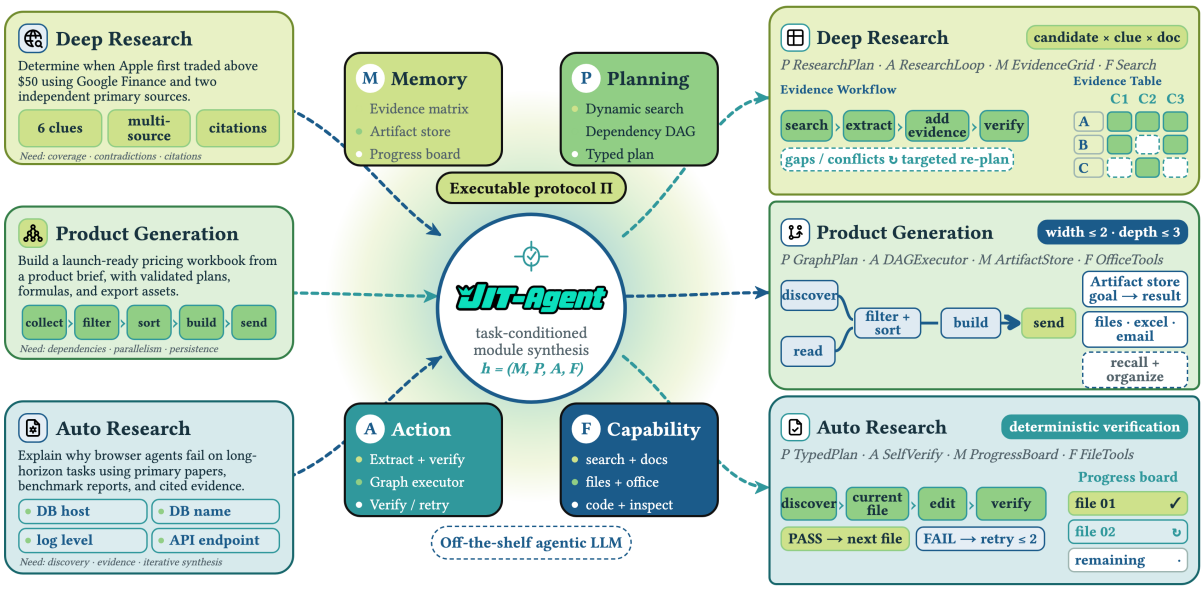


图 2: **JIT-Agent** 概览。给定一项任务，JIT-Agent 通过实例化（而非简单组合）记忆、规划、行动和能力四个模块，构造面向该问题的智能体支架。因此，不同的任务结构会产生不同的可执行协议和状态组织方式；图中以面向深度研究、产品生成和自主研究的专用支架为例加以说明。

动，以及何时触发验证或恢复 (Meng 等, 2026b; Zhou 等, 2026)。强大模型若置于不合适的记忆、规划器或行动协议之后也可能失败；反过来，强大支架也只有在模型能够理解并遵循它时才能释放能力 (Lee 等, 2026c; Lin 等, 2026)。因此，智能体智能并非仅是模型权重的属性，而是模型–支架组合的属性 (Lee 等, 2026c; Zhou 等, 2026)。所以，支架设计不是实现细节，而是决定智能体性能的首要因素 (Meng 等, 2026b; Lin 等, 2026)。

预先（AOT）支架 这一观察推动了越来越多关于测试时支架优化的研究 (Lee 等, 2026c; Lin 等, 2026)。近期系统根据轨迹和反馈优化支架代码、提示词、工具、记忆、技能或控制策略，在编码、终端使用、网页交互和通用智能体领域取得了显著提升 (Lee 等, 2026c; Lin 等, 2026)。尽管方法各异，其中许多都共享一种预先（AOT）假设：将支架视为一种持久制品，在经验流上对其进行优化，并期望所得制品能够泛化到未来的任务、领域或模型版本 (Lee 等, 2026c; Lin 等, 2026)。当部署分布稳定且同质时，这是一种强大的范式 (Lin 等, 2026; Zhang 等, 2025b)。然而，它仍要求优化循环在看到每个未来问题的确切结构之前，预先编译出一种用途广泛的支架 (Lee 等, 2026c)。

即时（JIT）支架。 我们转而采用一种**即时（JIT）的支架构建视角**。不同任务显然需要不同的支架先验。广域搜索任务可能受益于并行的证据探索 (Qin 等, 2025)；终端任务往往更适合精简的串行 ReAct 循环 (Yao 等, 2023a; Merrill 等, 2026)；深度研究任务需要面向检索证据的工作记忆 (Hu 等, 2025)；而 NL2Repo 风格的编码任务则天然以文件系统为媒介，由它存储补丁、测试、轨迹和代码仓库状态 (Yang 等, 2024; Ning 等, 2026)。简言之，合适的支架不仅依赖领域，也依赖具体实例 (Meng 等, 2026b; Zhou 等, 2026)。面对这些异质需求，优化单一 AOT 支架十分繁琐：必须搜索庞大的设计空间、积累足够的轨迹，然后寄希望于所得运行脚手架能够匹配下一个任务 (Lee 等, 2026c; Lin 等, 2026)。我们由此追问另一种可能性是否可行：**模型即支架（Model-as-a-Harness）**，即由训练后的元智能体当场生成任务特定的支架，再让任意智能体式大语言模型在该支架下执行。

JIT-Agent。 我们提出 JIT-Agent ，一个用于即时支架生成与演化的紧凑元智能体。[1] 推理时，JIT-Agent 接收任务描述、协议、可执行工具/技能注册表，以及一小段检索得到的既有支架上下文，并输出为当前任务量身定制的可执行支架。所得支架封装一个现成的智能体式大语言模型来执行任务。

[1] JIT-Agent 延续了我们在智能体系统递归式自我设计与自我演化方面的长期探索。在处理完整支架之前，我们已经研究了单个支架组件层面的模块化设计与演化：MemEvolve (Zhang 等, 2025b) 研究记忆，TodoEvolve (Liu 等, 2026a) 研究规划。JIT-Agent 是这一进程的统一延续，将组件级自我设计扩展到完整的运行支架。

随着反馈和轨迹不断积累，JIT-Agent 可以修订支架并更新支架档案，从而在生成器自身保持固定的同时实现测试时支架演化。训练这样的系统会面临三个挑战：❶ **适应性**，即如何使生成的支架与任务匹配；❷ **可靠性**，即如何确保行为可执行，并在合成失败时恢复 (Pan 等, 2026a; Ning 等, 2026)；以及❸ **可演化性**，即如何将执行反馈转化为更强的未来支架 (Lee 等, 2026c; Lin 等, 2026)。这三项要求共同定义了我们所称的**支架智能**：

支架智能

支架智能是构建和完善模型行动所依托之运行脚手架的能力。它具有三个定义性属性：**适应性**，使支架与任务和骨干模型相匹配；**可靠性**，产生可执行行为，并在合成失败时恢复；以及**可演化性**，将执行反馈转化为更强的未来支架。对支架本身的智能，始于固定支架内的能力达到边界之处。

JIT-Agent 通过在协议所诱导的支架空间上采用三阶段训练方案来实现支架智能 (Meng 等, 2026b; Zhou 等, 2026)。首先，我们将智能体支架分解为记忆、规划、行动和能力编排模块组成的四元组，并构建 HarnessFactory：一个在此通用接口下汇集代表性运行脚手架的统一代码库 (Hu 等, 2025; Erdogan 等, 2025a; Yao 等, 2023a; Xu 和 Yan, 2026)。秉持代码承载支架的观点 (Ning 等, 2026; Pan 等, 2026a)，这种表示使支架生成成为可能：JIT-Agent 生成结构化的可执行模块，而非不受约束的智能体程序。随后，❶ **阶段 I** 从教师生成且符合协议的样例中学习任务条件化定制，使生成器在共享设计空间上具备适应性。其次，❷ **阶段 II** 将失败的生成转换为有界修复轨迹，把编译器错误、接口不匹配和运行时故障转化为可靠恢复的监督信号。最后，❸ **阶段 III** 引入演化式组解耦策略优化（Evo-GDPO），优化 JIT-Agent 以提出超越当前档案前沿的支架，同时分别对奖励、延迟和成本进行归一化。它将测试时支架改进转化为一种训练所得能力，而非外部搜索启发式，从而赋予系统可演化性。通过这条流水线，我们训练得到 **JIT-Agent-27B**：一个能够定制、修复和演化任务特定支架的**支架智能模型**，将支架工程从 AOT 制品转向原生 JIT 合成。

实验结果。 结果表明，支架智能既能提升高效的骨干模型，也能进一步扩展已有强大模型的能力。配备 JIT-Agent 的 DeepSeek-V4-Flash 在 DeepSearchQA（+9.1）、PinchBench（+8.7）和 OdysseyBench（+4.3）上超过 GPT-5.6。对于本已强大的 GLM-5.2，JIT-Agent 进一步抬高了性能上限，在 xBench-DS（+12.0）和 AgentIF（+6.9）上取得了大幅绝对提升。除这些突出的结果之外，JIT-Agent 生成的支架在性能上可与 OpenCode、Claude Code 等成熟智能体运行时竞争，并在 DeepSeek V4、Mimo-V2.5 和 Qwen3.6 的不同规模变体上带来一致收益。

贡献。 我们的贡献总结如下：

- **支架智能**。我们将模型即支架形式化为一种新的智能体改进范式，并将支架智能定义为通过学习获得的、构建、修复和演化智能体运行脚手架的能力。
- **实用方案**。我们提出 JIT-Agent 及其三阶段训练流水线，将四模块可执行支架协议、HarnessFactory、定制学习、反馈驱动修复，以及用于测试时支架演化的 Evo-GDPO 结合起来。
- **实验验证**。我们在异质智能体基准和模型骨干上验证了 JIT-Agent，结果表明 JIT 生成的支架能够提升原始智能体、与强大的固定支架竞争，并推动骨干模型–支架组合迈向更优的成本–性能前沿。

2 相关工作

支架工程与模块化。 支架工程仍呈现出显著的多样性，涵盖 Claude Code、Codex、Hermes 和 OpenClaw 等编码及通用运行时 (Anthropics, 2025; OpenAI, 2025; Nous Research, 2026; OpenClaw Contributors, 2026)，具身系统 (Zhang 等, 2026e; Lee 等, 2026b)，搜索智能体 (Apodex Team, 2026; Qin 等, 2025)，以及 Mem0、EverMemOS 和 MemEvolve 等以记忆为中心的支架 (Chhikara 等, 2025; Hu 等, 2026a; Zhang 等, 2025b)。在这些多样形式之下，支架是决定智能体观察、记忆、规划、调用、验证和执行内容的运行层，因此能力是模型–支架组合的属性，而不只是模型权重的属性 (Meng 等, 2026b; Zhou 等, 2026; Pan 等, 2026a; Lee 等, 2026c)。反复出现的设计选择包括：**(i) 记忆**，负责整理工作上下文并保存跨会话经验 (Hu 等, 2026b, 2024a; Yan 等, 2025; Zhang 等, 2025c)；**(ii) 规划**，形成并修订子目标以更新智能体的行动方向 (Cao 等, 2025; Erdogan 等, 2025b; Jin 等, 2025; Qin 等, 2025)；**(iii) 控制循环与目标管理**，执行目标、监控进展、反思结果并从失败中恢复 (Yao

表 1: **支架优化范式**。构建方式区分了通过预先搜索得到的支架、随后利用测试时反馈编辑的预先支架，以及为每项任务即时生成的支架。实例合成、支架模型、学习式修复和在线演化分别表示某种方法是否直接合成实例特定支架、训练生成器、从失败的执行轨迹中学习修复，以及在部署后继续改进。

Method	Construction	Instance synthesis	Harness model	Learned repair	Online evolution
AutoHarness (Lou 等, 2026)	AOT (search)	✗	✗	✗	✗
Meta-Harness (Lee 等, 2026c)	AOT (search)	✗	✗	✗	✗
AHE (Lin 等, 2026)	AOT (search)	✗	✗	✗	✗
Adaptive AH (Liu 等, 2026c)	AOT (test-time editing)	✗	✗	✗	✓
TTHE (Nie 等, 2026b)	AOT (test-time editing)	✗	✗	✗	✓
RHI (Lee 等, 2026a)	AOT (test-time editing)	✗	✗	✗	✓
Harness-R1 (Shao 等, 2026)	AOT (test-time editing)	✗	✓	✓	✓
JIT-Agent (ours)	**JIT**	✓	✓	✓	✓

等, 2023a; Shinn 等, 2023; Madaan 等, 2023; Sun 等, 2023)；**(iv) 工具分配**，从大型 API 空间中选择可调用资源并安排其顺序 (Qin 等, 2023; Yang 等, 2026; Qian 等, 2025; Wang 等, 2025a)；以及**(v) 技能编排**，从不断扩大的技能库中发现、检索、组合并委派可复用流程 (Xu 和 Yan, 2026; Zheng 等, 2025; Li 等, 2026a; Zhou 等, 2026)。因此，模块化已成为该领域的通用设计语言：尽管表示方式各异，显式接口仍使支架组件可以独立替换和优化 (Chen 等, 2026b; Ning 等, 2026; Pan 等, 2026a; Lin 等, 2026; Lee 等, 2026c)。例如，HarnessX 将运行时分解为提示词、工具、记忆和控制流，而 Code-as-Agent-Harness 则将其组织为接口、机制和多智能体扩展层 (Chen 等, 2026b; Ning 等, 2026)。因此，我们采用紧凑分解 $\mathrm{h} = (\mathrm{M}, \mathrm{P}, \mathrm{A}, \mathrm{F})$，四个模块分别构建记忆视图、形成下一条指令、推进行动循环，以及编排工具与技能；这既提供了最小可执行契约，也保持了每个模块内部的可扩展性 (Hu 等, 2026b; Cao 等, 2025; Yao 等, 2023a; Xu 和 Yan, 2026)。

支架优化。 完成模块化之后，支架构建便从一种固定的工程选择转变为针对可执行制品的优化问题 (Khattab 等, 2023; Yuksekgonul 等, 2024; Hu 等, 2024b; Shang 等, 2024)。优化范围已从提示词和声明式语言模型流水线扩展到工作流图、角色分配和完整支架代码，使反馈能够同时修订局部指令和全局控制结构 (Khattab 等, 2023; Zhang 等, 2024; Hu 等, 2024b; Shang 等, 2024; Lee 等, 2026c)。**闭环演化。**在支架层面，近期方法大多实现了“诊断–提出–验证”循环：执行轨迹暴露行为弱点，候选修改改变提示词、记忆、工具、中间件或控制流，而验证或偏好信号决定哪些修改得以保留 (Lee 等, 2026c; Lin 等, 2026; Zhang 等, 2026b; Pan 等, 2026b; Chen 等, 2026b)。近期工作进一步将这一循环扩展到递归式、层次化和协作式自我改进，同时约束逐组件演化以增强迁移能力 (Lee 等, 2026a; Zhou, 2026; Nie 等, 2026a; Zhang 等, 2026c)。其监督信号涵盖文本梯度、基于轨迹的可观测性、回归测试、回顾式自我偏好以及有类型原语替换，但共同产物都是一种能够持续改进的可执行运行脚手架 (Yuksekgonul 等, 2024; Lin 等, 2026; Zhang 等, 2026b; Pan 等, 2026b; Chen 等, 2026b)。**AOT 与 JIT。**尽管这些支架可以迁移，但此类方法主要针对累积的经验分布优化一种持久制品，因此相对于新实例的确切结构而言属于预先优化 (Zhang 等, 2024, 2025a; Yuan 等, 2024; Chen 等, 2023)。我们则将支架搜索摊销进 JIT-Agent：生成器在推理时合成任务条件化支架，并能依据执行反馈进一步修订支架，从而将优化从反复制品搜索转向学习所得的即时构建 (Meng 等, 2026b; Zhou 等, 2026; Ning 等, 2026; Pan 等, 2026a; Xu 和 Yan, 2026)。表 1 明确呈现了这一区别。

3 统一支架代码库

3.1 模块化支架设计空间

智能体支架是将基础模型转变为闭环智能体的运行层 (Meng 等, 2026a; Zhou 等, 2026)：它决定保留先前交互中的哪些内容 (Hu 等, 2026b)、如何形成中间指令 (Cao 等, 2025)、每个阶段开放哪些外部能力 (Xu 和 Yan, 2026; Yang 等, 2026)，以及如何推进控制流程。因此，支架生成定义在程序

而非不受约束的文本之上。我们区分原始生成空间 $\mathcal{G}$、语法有效的支架 $\mathcal{H}^{\mathrm{syn}}$、符合协议的支架 $\mathcal{H}_{\Pi}$ 及其可执行子集 $\mathcal{H}_{\Pi}^{\mathrm{exec}}$，并有 $\mathcal{G} \supseteq \mathcal{H}^{\mathrm{syn}} \supseteq \mathcal{H}_{\Pi} \supseteq \mathcal{H}_{\Pi}^{\mathrm{exec}}$。固定协议 Π 规定模块模式与接口、模块生命周期、验证规则及共享执行语义。它消除了语言和运行时中的偶然差异，同时保留了区分现有支架的主要运行选择 (Wu 等, 2023; Microsoft, 2024; LangGraph, 2025; Luo 等, 2025)。

支架可组合性

可组合性已成为现代支架设计中的首要关切。例如，DeepSeek Harness 采用“一切皆插件”架构，将运行时表示为通过显式依赖关系连接的模块 (Shi 等, 2026)。可组合性对于 JIT 生成同样至关重要：它将支架构建从不受约束的程序合成转变为在有类型、可重组设计空间上的装配。

JIT-Agent 通过四个可互操作模块明确表达这些选择：如何压缩历史、如何形成局部意图、如何编排工具与技能，以及如何推进控制流程。令 $\boldsymbol{\tau}$ 为任务，π_{ψ} 为冻结的骨干执行器，C_{τ} 为该任务可用的能力注册表（例如工具、API 和技能）。运行支架 h 并由 π_{ψ} 执行，会产生闭环轨迹

$$\boldsymbol{\xi} \sim \mathrm{Rollout}(\boldsymbol{\tau}, \pi_{\psi}, \mathrm{h}, C_{\tau}; \boldsymbol{\Pi}) = (\mathrm{s}_1, e_1, \mathrm{o}_1, \ldots, \mathrm{s}_T, e_T, \mathrm{o}_T), \tag{1}$$

其中，$\mathrm{s}_t \in \mathcal{S}$ 是所维护的控制器状态，e_t 是发出的工具调用或终止输出，$\mathrm{o}_t \in \mathcal{O}$ 是由此产生的观测，而 T 是受协议或预算约束的停止时刻。我们的关键假设是，每个 $\mathrm{h} \in \mathcal{H}_{\Pi}$ 都允许进行如下模块化分解

$$\mathrm{h} = (\mathrm{M}, \mathrm{P}, \mathrm{A}, \mathrm{F}) \in \mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}, \tag{2}$$

其中，M、P、A 和 F 分别表示记忆、规划、行动和能力编排模块，$\mathfrak{M}$、$\mathfrak{P}$、$\mathfrak{A}$ 和 $\mathfrak{F}$ 是它们与协议兼容的实现空间。该元组遵循这一概念分解；运行时的依赖顺序为 $\mathrm{M} \to \mathrm{P} \to \mathrm{F} \to \mathrm{A}$。所有模块都基于同一个冻结的 π_{ψ} 运行，下文省略这一共同的运行时依赖。

定位

Codex、Claude Code 和 DeepSeek Harness 等生产级支架所提供的机制远比我们的四模块实例丰富 (Shi 等, 2026; Anthropics, 2025; OpenAI, 2025)。这种差距是有意为之。本工作并非要求模型复现整个生产运行时，而是要确立一个更基础的结论：即便是即时生成的紧凑支架，也能带来可观收益。此外，我们认为 JIT 范式与生产级支架工程相互补充。随着模型越来越多地参与自身运行时的设计和修订，支架智能的训练方案在更大规模上也将十分重要。我们将当前设计空间视为一个起点。

该协议同时维护不可变的事件历史 $\boldsymbol{\xi}_{<t}$ 和可变的控制器状态 s_t，二者按如下方式交互

$$\begin{aligned}
\mathrm{v}_t &= \mathrm{M}(\boldsymbol{\xi}_{<t}, \mathrm{s}_t) \in \mathcal{V}, && \text{history} \to \text{view}, && (3)\\
\mathrm{d}_t &= \mathrm{P}(\boldsymbol{\tau}, \mathrm{s}_t, \mathrm{v}_t) \in \mathcal{D}_{\mathrm{dir}}, && \text{view} \to \text{local directive}, && (4)\\
C_t &= \mathrm{F}(C_{\tau}, \mathrm{s}_t, \mathrm{v}_t, \mathrm{d}_t) \subseteq C_{\tau}, && \text{directive-conditioned capability orchestration}, && (5)\\
(\mathrm{s}_{t+1}, e_t) &= \mathrm{A}(\mathrm{s}_t, \boldsymbol{\tau}, \mathrm{v}_t, \mathrm{d}_t, C_t) \in \mathcal{S} \times \mathcal{A}, && \text{control update and action emission}, && (6)
\end{aligned}$$

其中，$\mathcal{V}$ 和 $\mathcal{D}_{\mathrm{dir}}$ 分别是视图空间和指令空间，$\mathcal{A} = \mathcal{U} \sqcup \mathcal{Y}$ 是可执行调用与终止输出的不交并。注册表 C_{τ} 包含可调用工具（例如 bash 工具、API 和 MCP (Sarkar 和 Sarkar, 2025)）以及更高层级的智能体技能 (Xu 和 Yan, 2026; ClaudeCode, 2025)。不含显式规划器的支架通过空指令 $\mathrm{d}_{\emptyset} \in \mathcal{D}_{\mathrm{dir}}$ 保持类型一致，该指令由 $\mathrm{P}_{\emptyset}$ 在每一步返回。也就是说，记忆根据已发生的历史构建视图，规划将该视图转换为局部指令，能力编排激活相关外部工具或智能体技能，而行动模块使用组装后的上下文，既更新控制器状态，也发出下一项行动。随后，内核按如下方式解释所发出的行动

$$\mathrm{o}_t = \begin{cases} \mathrm{Exec}(e_t; C_t), & e_t \in \mathcal{U}, \\ \bot, & e_t \in \mathcal{Y}, \end{cases} \qquad \boldsymbol{\xi}_{\le t} = \boldsymbol{\xi}_{<t} \oplus (\mathrm{s}_t, e_t, \mathrm{o}_t), \tag{7}$$

其中，$\mathrm{Exec} : \mathcal{U} \times 2^{C_{\tau}} \to \mathcal{O}$ 是共享执行内核，$\bot \in \mathcal{O}$ 是终止空观测，$\oplus$ 将一个事件追加到轨迹中。执行从 $\boldsymbol{\xi}_{<1} = \emptyset$ 和协议定义的 $\mathrm{s}_1 \in \mathcal{S}$ 开始，在 $e_t \in \mathcal{Y}$ 时终止，得到 $y = e_T \in \mathcal{Y}$。这一分解将原本异质的程序转化为 $\mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}$ 中可比较的坐标：

表 2: 种子库 $\mathcal{B}_0$：13 个实例化四模块协议 Π 的手工编写支架。每一行都是一个完整支架；各列依次对应记忆、规划、行动和能力编排的概念顺序。

Harness	♣ Memory $M \in \mathfrak{M}$	♦ Planning $P \in \mathfrak{P}$	♥ Action $A \in \mathfrak{A}$	♠ Capability Orchestration $F \in \mathfrak{F}$
ReAct [83]	FullHistory	No explicit planner	ReAct	Full registry
Plan-and-Execute [16]	FullHistory	Linear roadmap	ReAct	Full registry
ReSum [77]	ReSum memory	No explicit planner	ReAct	Full registry
Flash-Searcher [60]	FullHistory	DAG planning	ReAct	Full registry
GAM [79]	GAM retrieval	DAG	ReAct	Full registry
MemoBrain [59]	Reasoning graph	No explicit planner	Marker-guided execution	Full registry
AggAgent [32]	Isolated rollout histories	No explicit planner	Multi-rollout aggregation	Full registry
OAgent [100]	Coordinator history	No explicit planner	Ensemble voting	Full registry
AgentFold [84]	AgentFold memory	DAG	ReActFold	Full registry
HiAgent [22]	Hierarchical memory	No explicit planner	ReAct	Full registry
DeepAgent [34]	Three-tier memory	No explicit planner	Marker-guided execution	Tool search
ROMA [1]	Context isolation	Atomizer + DAG	Recursive execution	Full registry
AOrchestra [63]	Context isolation	Atomizer + DAG	Recursive execution	Agent Delegation

- 经典 **ReAct** 可写为 $\mathrm{h}_{\mathrm{ReAct}} = (\mathrm{M}_{\mathrm{full}}, \mathrm{P}_{\emptyset}, \mathrm{A}_{\mathrm{react}}, \mathrm{F}_{\mathrm{all}})$，其中 $\mathrm{A}_{\mathrm{react}}$ 是标准 ReAct 循环，$\mathrm{P}_{\emptyset}$ 表示不存在显式规划器，$\mathrm{M}_{\mathrm{full}}$ 在不进行上下文管理的情况下保留运行历史，而 $\mathrm{F}_{\mathrm{all}}$ 则开放完整的工具/技能注册表。
- Codex (OpenAI, 2025) 和 OpenCode (OpenCode-AI, 2025) 等工程化 **ReAct** 变体仍符合相同的运行脚手架：$(\mathrm{M}_{\mathrm{compact}}, \mathrm{P}_{\mathrm{todo}}, \mathrm{A}_{\mathrm{react}}, \mathrm{F}_{\mathrm{all}})$。其中，行动内核仍采用 ReAct 风格，但 $\mathrm{M}_{\mathrm{compact}}$ 会在接近上下文上限时压缩历史，$\mathrm{P}_{\mathrm{todo}}$ 则维护显式的任务待办列表。
- **递归智能体。** ROMA (Alzu'bi 等, 2026)、AOrchestra (Ruan 等, 2026) 和递归语言模型（RLM）(Zhang 等, 2026a) 等递归架构，同样可由 $\mathrm{h}_{\mathrm{rec}} = (\mathrm{M}_{\mathrm{subproblem}}, \mathrm{P}_{\mathrm{decomp}}, \mathrm{A}_{\mathrm{rec}}, \mathrm{F}_{\mathrm{route}})$ 之类的选择来刻画。在此类系统中，主编排器可通过 $\mathrm{A}_{\mathrm{rec}}$ 生成子智能体、在 $\mathrm{P}_{\mathrm{decomp}}$ 中维护待办列表式分解、在 $\mathrm{M}_{\mathrm{subproblem}}$ 中保持智能体上下文相互隔离，并通过 $\mathrm{F}_{\mathrm{route}}$ 为每个子智能体分配工具或技能。

在这一模块化设计空间的基础上，我们接下来介绍 HarnessFactory；它用于检验 $\mathcal{H}_{\Pi}$ 的表达能力，并为 JIT-Agent 的元支架设计提供多样化素材。

3.2 HarnessFactory

在共享协议 Π 和通用内核之下，HarnessFactory 重新实现了 13 种具有代表性的当代智能体运行脚手架：ReAct (Yao 等, 2023b)、Plan-and-Execute (Erdogan 等, 2025b)、ReSum (Wu 等, 2026)、Flash-Searcher (Qin 等, 2025)、General Agentic Memory (GAM) (Yan 等, 2025)、MemoBrain (Qian 等, 2026)、AggAgent (Lee 等, 2026d)、OAgent (Zhu 等, 2025)、AgentFold (Ye 等, 2025)、HiAgent (Hu 等, 2024a)、DeepAgent (Li 等, 2026b)、ROMA (Alzu'bi 等, 2026)，以及 AOrchestra (Ruan 等, 2026)。它们提供异质的记忆、规划、行动和能力编排策略。因此，种子库 $\mathcal{B}_0$ 包含 $K_0 = 13$ 个与协议兼容的支架。表 2 给出了完整清单。

该种子库具有双重作用：从 $\mathcal{B}_0$ 采样的支架为阶段 I 的合成提供锚点，而之后的档案状态则提供用于评估新设计的先验种群。随着系统运行，$\mathcal{B}_0$ 扩展为 $\mathcal{B}_n \supseteq \mathcal{B}_0$；其中每个条目都将保留的支架与其任务，以及观测到的奖励、延迟和成本关联起来；这一演化过程将在节 4.3 中形式化。

4 训练流水线

训练遵循 JIT-Agent 的推理时生命周期：首先合成一个任务条件化支架，随后修复不稳定的生成结果，最后通过更优的存档状态在线学习改进。设 $\boldsymbol{\tau} \sim \mathcal{D}_{\mathrm{task}}$ 表示从训练分布中抽取的任务，$\mathcal{C}_\tau$ 表示其能力注册表，$\mathcal{E}_\tau$ 表示来自支架库的小规模参考上下文。我们将生成上下文定义为 $\mathrm{c}_\tau = (\boldsymbol{\tau}, \boldsymbol{\Pi}, \mathcal{C}_\tau, \mathcal{E}_\tau)$，并从 $p_\theta(\mathrm{h} \mid \mathrm{c}_\tau)$ 中采样支架。随后，一个冻结的执行器 $\pi_\psi \sim \mathcal{M}$ 运行每个支架，以进行验证和效用度量。统一目标为

$$\theta^\star = \arg\max_{\theta} \mathbb{E}_{\substack{\boldsymbol{\tau}\sim\mathcal{D}_{\mathrm{task}},\, \pi_\psi\sim\mathcal{M}, \\ \mathrm{h}\sim p_\theta(\cdot\mid \mathrm{c}_\tau)}} \big[U(\boldsymbol{\tau}, \pi_\psi, \mathrm{h})\big], \tag{8}$$

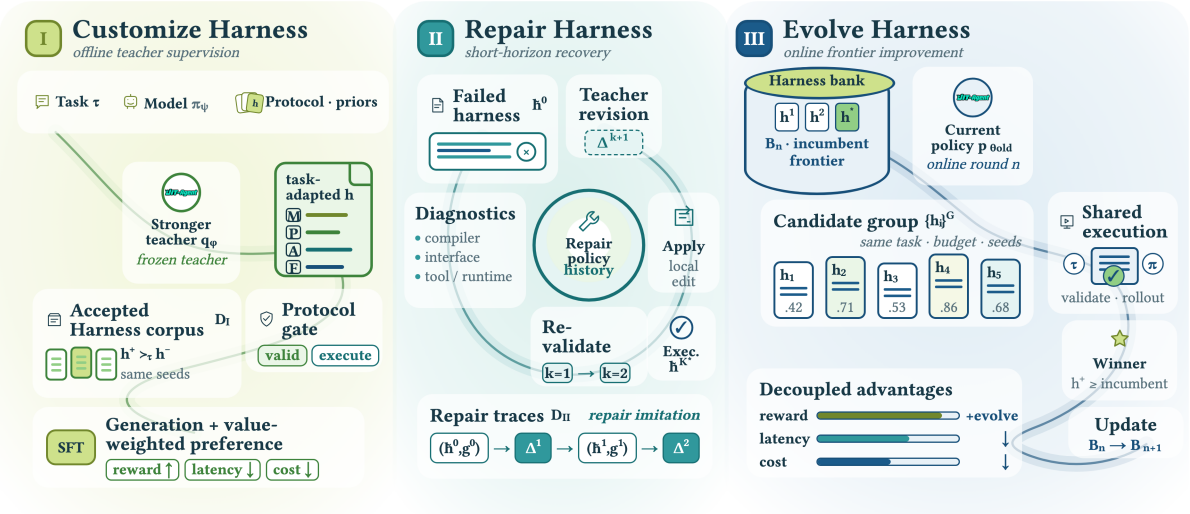


图 3: **JIT-Agent 的训练流水线**。阶段 I 学习任务条件化的支架定制。阶段 II 将失败的支架和执行诊断转换为有界修复轨迹。阶段 III 通过将候选支架与当前种子库比较来执行在线演化，优化解耦的奖励、延迟和成本优势，并保留能够改进前沿的设计。

其中，U 使用任务奖励、时延和货币成本来评估由模型–支架组合产生的轨迹。由于 p_θ 在 $\mathcal{G}$ 中进行生成，而非通过构造直接保证可执行性，因此协议验证器会检查每个输出。我们使用 $\mathrm{Valid}_\Pi(\mathrm{h};\boldsymbol{\tau},\pi_\psi,C_\tau)\in\{0,1\}$ 表示其有效性标志；检查失败时还会返回一份结构化诊断报告。我们通过阶段 I 的离线监督、阶段 II 的修复轨迹学习和阶段 III 的在线策略改进来实例化式 (8)。

4.1 阶段 I：定制支架

数据准备。 阶段 I 使用一个冻结的、更强的教师模型 q_ϕ，在固定的四模块协议下合成适配任务的支架。对于每个任务，从种子库中与任务类型匹配的子集 $\mathcal{B}_0^{(d(\boldsymbol{\tau}))}$ 采样三个参考支架，

$$\mathcal{E}_\tau=\{\mathrm{h}^{(1)},\mathrm{h}^{(2)},\mathrm{h}^{(3)}\}\sim\mathrm{Sample}_3(\mathcal{B}_0^{(d(\boldsymbol{\tau}))}),\qquad \mathrm{h}^{\mathrm{teach}}\sim q_\phi(\cdot\mid \mathrm{c}_\tau), \tag{9}$$

其中，$d(\boldsymbol{\tau})$ 表示任务类型。教师模型接收任务、协议、能力注册表和采样得到的支架。只有通过协议验证和执行检查的生成结果才会被保留，由此得到

$$\mathcal{D}_\mathrm{I}=\Big\{(\boldsymbol{\tau},\pi_\psi,C_\tau,\mathcal{E}_\tau,\mathrm{h}^{\mathrm{teach}})\;\Big|\;\mathrm{Valid}_\Pi(\mathrm{h}^{\mathrm{teach}};\boldsymbol{\tau},\pi_\psi,C_\tau)=1\Big\}. \tag{10}$$

阶段 I 的训练任务来自多个不同来源，包括 (Wu 等, 2025; Shi 等, 2025; Zhang 等, 2026d; Bai 等, 2026; Burgess 等, 2026; Song 等, 2026)，以及一部分合成任务。

训练设置。 阶段 I 使用两个相互耦合的目标。第一个是在已接受的教师模型生成结果上采用标准监督微调目标：

$$\mathcal{L}_\mathrm{I}^{\mathrm{gen}}(\theta)=-\,\mathbb{E}_{(\boldsymbol{\tau},\pi_\psi,C_\tau,\mathcal{E}_\tau,\mathrm{h}^{\mathrm{teach}})\sim\mathcal{D}_\mathrm{I}}\sum_{j=1}^{|\mathrm{h}^{\mathrm{teach}}|}\log p_\theta\big(y_j^{\mathrm{teach}}\mid y_{<j}^{\mathrm{teach}},\mathrm{c}_\tau\big), \tag{11}$$

其中，y_j^{teach} 是第 j 个目标词元。这使 JIT-Agent 学会将一个任务和小规模参考上下文直接映射为符合协议的支架。仅满足协议仍然不够：可执行支架在任务奖励、时延和货币成本方面仍可能存在显著差异。因此，我们在相同骨干模型和评估随机种子下比较候选方案；只有当奖励提升、两个效率维度均未下降且至少一个效率维度得到严格改善时，才保留该偏好关系。

$$\begin{aligned}&\mathrm{h}^+\succ_\tau\mathrm{h}^-\iff r^+>r^-\;\wedge\;\ell^+\le\ell^-\;\wedge\;\kappa^+\le\kappa^-\;\wedge\;\big(\ell^+<\ell^-\;\vee\;\kappa^+<\kappa^-\big),\\&\Delta_{\mathrm{val}}(\boldsymbol{\tau};\mathrm{h}^+,\mathrm{h}^-)=\alpha_r(r^+-r^-)+\alpha_\ell[\ell^--\ell^+]_+ +\alpha_\kappa[\kappa^--\kappa^+]_+,\end{aligned} \tag{12}$$

其中，$(r^{\pm}, \ell^{\pm}, \kappa^{\pm})$ 是 $(\mathrm{h}^+, \mathrm{h}^-)$ 在重复轨迹采样下的均值，$[x]_+ = \max(x, 0)$，而 $\alpha_r, \alpha_\ell, \alpha_\kappa \geq 0$ 控制三个价值差距。令 $\mathcal{D}_{\mathrm{I}}^{\mathrm{pref}}$ 收集由此得到的偏好元组。我们优化如下参考锚定目标：

$$\mathcal{L}_{\mathrm{I}}^{\mathrm{pref}}(\theta) = -\mathbb{E}_{(\boldsymbol{\tau}, \pi_\psi, C_\tau, \mathcal{E}_\tau, \mathrm{h}^+, \mathrm{h}^-) \sim \mathcal{D}_{\mathrm{I}}^{\mathrm{pref}}} \left[\Delta_{\mathrm{val}} \cdot \log \sigma \left(\beta_{\mathrm{pref}} \log \frac{p_\theta(\mathrm{h}^+ \mid \mathrm{c}_\tau)}{p_\theta(\mathrm{h}^- \mid \mathrm{c}_\tau)} - \beta_{\mathrm{pref}} \log \frac{p_{\mathrm{ref}}(\mathrm{h}^+ \mid \mathrm{c}_\tau)}{p_{\mathrm{ref}}(\mathrm{h}^- \mid \mathrm{c}_\tau)} \right) \right]. \quad (13)$$

这里，σ 是逻辑斯谛函数，$\beta_{\mathrm{pref}} > 0$ 控制偏好的尖锐程度，p_{ref} 是冻结的阶段 I SFT 检查点；在对数比值中，$\log p(\mathrm{h} \mid \cdot)$ 表示经过长度归一化的序列对数似然。阶段 I 的完整目标为 $\mathcal{L}_{\mathrm{I}}(\theta) = \mathcal{L}_{\mathrm{I}}^{\mathrm{gen}}(\theta) + \lambda_{\mathrm{pref}} \mathcal{L}_{\mathrm{I}}^{\mathrm{pref}}(\theta)$，其中 $\lambda_{\mathrm{pref}} \geq 0$。生成模仿建立符合协议的结构，而偏好学习则使模型偏向同时更有效且更高效的支架。

4.2 阶段 II：修复支架

数据准备。 阶段 I 针对协议合规性进行优化，但仍不能保证可执行性：一些生成的支架未能通过静态验证或运行时验证，因此无法进入 $\mathcal{D}_{\mathrm{I}}$。阶段 II 不丢弃这些失败样例，而是将其转化为修复监督。令

$$\mathcal{D}_{\mathrm{I}}^{\mathrm{fail}} = \left\{ (\boldsymbol{\tau}, \pi_\psi, C_\tau, \mathcal{E}_\tau, \widetilde{\mathrm{h}}^{(0)}, \mathrm{g}^{(0)}) \;\middle|\; \widetilde{\mathrm{h}}^{(0)} \notin \mathcal{H}_{\boldsymbol{\Pi}}^{\mathrm{exec}} \right\}, \quad (14)$$

其中，$\widetilde{\mathrm{h}}^{(0)}$ 是阶段 I 生成失败的支架，$\mathrm{g}^{(0)}$ 是其诊断报告，包含编译器错误、接口不匹配、工具调用失败和运行时异常。对于每个失败样例，教师模型提出结构化修订 $\Delta^{(k+1)} \in \mathcal{P}$；该修订来自补丁空间 $\mathcal{P}$，随后 Apply 以确定性方式将其应用到当前支架：

$$\widetilde{\mathrm{h}}^{(k)}, \mathrm{g}^{(k)} \longrightarrow \Delta^{(k+1)} \longrightarrow \widetilde{\mathrm{h}}^{(k+1)} = \mathrm{Apply}\left(\widetilde{\mathrm{h}}^{(k)}, \Delta^{(k+1)} \right), \qquad k \geq 0. \quad (15)$$

每次修订后，验证过程都会生成下一份报告 $\mathrm{g}^{(k+1)}$。我们只保留在两轮修复内变得可执行的轨迹。定义

$$K^\star = \min \left\{ k \in \{1, 2\} : \mathrm{Valid}_{\boldsymbol{\Pi}}(\widetilde{\mathrm{h}}^{(k)}; \boldsymbol{\tau}, \pi_\psi, C_\tau) = 1 \right\}, \quad (16)$$

仅当 $K^\star$ 有定义时，才构造阶段 II 的监督数据。这使语料库聚焦于真实且可局部恢复的失败，而非需要全面重新设计的失败。

训练设置。 随后，阶段 II 训练 JIT-Agent 模仿成功的修复转移，而非执行单次合成。对于每条保留的轨迹，令

$$\mathcal{D}_{\mathrm{II}} = \left\{ (\boldsymbol{\tau}, \pi_\psi, C_\tau, \mathcal{E}_\tau, \mathcal{R}_{K^\star}) \;\middle|\; (\boldsymbol{\tau}, \pi_\psi, C_\tau, \mathcal{E}_\tau, \widetilde{\mathrm{h}}^{(0)}, \mathrm{g}^{(0)}) \in \mathcal{D}_{\mathrm{I}}^{\mathrm{fail}},\ K^\star \text{ exists} \right\}, \quad (17)$$

其中，$\mathcal{R}_{K^\star} = \{(\widetilde{\mathrm{h}}^{(j)}, \mathrm{g}^{(j)}, \Delta^{\star(j+1)})\}_{j=0}^{K^\star - 1}$，并且每个 $\Delta^{\star(j+1)}$ 都是沿着一条最终变得可执行的修复轨迹所做的教师模型修订。修复目标以完整历史为条件，

$$\mathcal{L}_{\mathrm{II}}(\theta) = -\mathbb{E}_{\mathcal{D}_{\mathrm{II}}} \sum_{k=0}^{K^\star - 1} \log p_\theta \left(\Delta^{\star(k+1)} \;\middle|\; \mathrm{c}_\tau, \{(\widetilde{\mathrm{h}}^{(j)}, \mathrm{g}^{(j)})\}_{j=0}^{k} \right). \quad (18)$$

由于 $K^\star \leq 2$，由此得到的监督恰好针对部署时至关重要的短时域修复情形：给定一个近乎正确但不稳定的支架，JIT-Agent 学会利用执行反馈产生少量高影响力的修订，以恢复符合协议的执行。

4.3 阶段 III：学习演化支架

阶段 III 将测试时支架演化本身视为一种可训练能力。其目标并非仅仅恢复或模仿此前观察到的支架设计，而是优化 JIT-Agent ，使其能够在测试时反复提出超越既有设计的支架，让这些支架成为未来更强的参考，并持续推进支架能力前沿。受 (Liu 等, 2026b) 启发，我们将由此得到的在线目标称为**演化式组解耦策略优化（Evo-GDPO）**。

数据准备。 阶段 III 的训练样例由三个同步来源构建：一个任务实例、一小组此前的高质量支架设计，以及在同一任务下在线收集的新执行反馈。具体而言，在第 n 轮在线训练中，我们采样任务 $\boldsymbol{\tau}$，检索小规模参考集 $\mathcal{E}_{\tau,n}$；该参考集来自当前支架库 $\mathcal{B}_n$，随后我们构造 $\mathrm{c}_{\tau,n} = (\boldsymbol{\tau}, \boldsymbol{\Pi}, C_\tau, \mathcal{E}_{\tau,n})$。模型提出候选方案，并在相同的冻结执行器 $\pi_\psi \sim \mathcal{M}$、预算和评估随机种子下，将其与这些既有设计一同执行。在 $\mathcal{E}_{\tau,n}$ 中，奖励最高的支架作为唯一的当前最优方案；若奖励相同，则依次选择时延更低、成本更低的支架。其统计量记为 (b_r, b_ℓ, b_κ)。

训练设置。 给定上述上下文，Evo-GDPO 首先从当前策略中采样一组候选支架：

$$\mathrm{h}_i \overset{\text{i.i.d.}}{\sim} p_{\theta_{\text{old}}}(\cdot \mid \mathrm{c}_{\tau,n}), \qquad i = 1, \dots, G, \tag{19}$$

其中，$G > 1$ 是组大小，θ_{old} 是轨迹采样策略快照。每个候选方案在执行前都要经过验证；若某个候选方案在有界修复后仍然无效，则赋予其最低任务奖励，同时其修复时延和成本仍计入效率度量。对于可执行的候选方案，r_i、$\bar{\ell}_i$ 和 $\bar{\kappa}_i$ 分别表示多次轨迹采样下的奖励、平均时延和平均货币成本。奖励通道是主要通道，而效率通道仅在候选方案保持当前最优方案奖励时激活。

$$\begin{aligned} R_i^{\text{rew}} &= r_i + \lambda_{\text{evo}}\,[r_i - b_r]_+, \\ R_i^{\text{lat}} &= \mathbb{I}[r_i \geq b_r]\,[b_\ell - \bar{\ell}_i]_+, \\ R_i^{\text{cost}} &= \mathbb{I}[r_i \geq b_r]\,[b_\kappa - \bar{\kappa}_i]_+. \end{aligned} \tag{20}$$

这里，$\lambda_{\text{evo}} \geq 0$ 控制奖励加成，$\mathbb{I}[\cdot]$ 是指示函数。Evo-GDPO 在合并三个信号之前分别对其进行归一化，避免它们的数值尺度相互压制。第二次批次级归一化用于稳定跨任务优化。

$$\begin{aligned} A_i^m &= \frac{R_i^m - \mathrm{mean}(\{R_j^m\}_{j=1}^G)}{\mathrm{std}(\{R_j^m\}_{j=1}^G) + \varepsilon_{\text{num}}}, \qquad m \in \{\text{rew}, \text{lat}, \text{cost}\}, \\ A_i^\Sigma &= w_{\text{rew}} A_i^{\text{rew}} + w_{\text{lat}} A_i^{\text{lat}} + w_{\text{cost}} A_i^{\text{cost}}, \quad w_{\text{rew}} > w_{\text{lat}} + w_{\text{cost}}, \\ \widehat{A}_i^\Sigma &= \frac{A_i^\Sigma - \mathrm{mean}_{\text{batch}}(A^\Sigma)}{\mathrm{std}_{\text{batch}}(A^\Sigma) + \varepsilon_{\text{num}}}. \end{aligned} \tag{21}$$

这些非负权重之和为一，而上述不等式确保任务奖励占据主导；$\varepsilon_{\text{num}} > 0$ 是上述两个分母中的数值稳定项。最终的策略更新采用由该聚合优势驱动的 PPO 风格裁剪目标。相较于标准的 GRPO 风格训练，模型获得奖励并不只是因为其在采样组内表现良好，还因为它能够超越既有支架设计，并在保持奖励质量的前提下以更高效率实现这一点。

$$\begin{aligned} \mathcal{L}_{\text{III}}^{\text{Evo-GDPO}}(\theta) = &-\mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathrm{h}_i|}\sum_{j=1}^{|\mathrm{h}_i|}\min\Big(\rho_{i,j}(\theta)\widehat{A}_i^\Sigma, \mathrm{clip}(\rho_{i,j}(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}})\widehat{A}_i^\Sigma\Big)\right] \\ &+ \beta_{\text{KL}}\,\mathbb{E}\big[\mathrm{KL}(p_\theta \,\|\, p_{\text{ref}})\big], \\ \rho_{i,j}(\theta) = &\frac{p_\theta(y_{i,j} \mid y_{i,<j}, \mathrm{c}_{\tau,n})}{p_{\theta_{\text{old}}}(y_{i,j} \mid y_{i,<j}, \mathrm{c}_{\tau,n})}. \end{aligned} \tag{22}$$

该期望覆盖采样任务、冻结执行器以及从 $p_{\theta_{\text{old}}}$ 中采样的候选组；$|\mathrm{h}_i|$ 是支架的词元长度，$\epsilon_{\text{clip}} > 0$ 是裁剪半径，$\beta_{\text{KL}} \geq 0$，p_{ref} 是用于词元级 KL 惩罚的冻结阶段 II 检查点。优化后，我们保守地更新支架库：只有当某个候选方案达到或超过当前奖励前沿，并且进一步严格改善至少一个前沿维度—奖励本身、时延或成本—时，才保留该候选方案。在训练期间，这些反馈会同时更新 θ 和 $\mathcal{B}_n$；在部署时，θ 保持冻结。

5 推理架构

JIT-Agent 支持两种推理模式：静态推理和流式推理；二者的区别在于，任务结束后是丢弃经验，还是保留经验以支持后续任务。

静态推理。 在该模式下，我们引入一种轻量级测试时扩展方式：JIT-Agent 并行生成 N 个支架，从中选出一个，并且只执行所选支架。该方式无需增加环境轨迹采样次数，即可提高候选方案的多样性。所选支架遵循上文所述的相同验证与有界修复流程。

流式推理。 流式推理模式旨在跨越一系列任务传递有用经验。对于第 n 个任务 $\boldsymbol{\tau}_n$，JIT-Agent 从当前支架库 $\mathcal{B}_n$ 中检索参考，生成并选择支架 $\mathrm{h}_n^\dagger$，然后执行一次。随后，所得环境反馈仅用于判断该经验是否应当更新支架库：

$$\begin{aligned} \boldsymbol{\xi}_n &\sim \mathrm{Rollout}(\boldsymbol{\tau}_n, \pi_\psi, \mathrm{h}_n^\dagger, C_{\tau_n}; \boldsymbol{\Pi}), \\ \mathrm{m}_n &= \mathrm{Eval}(\boldsymbol{\xi}_n), \\ \mathcal{B}_{n+1} &= \mathrm{Update}_{\mathrm{III}}(\mathcal{B}_n; \boldsymbol{\tau}_n, \mathrm{h}_n^\dagger, \mathrm{m}_n), \\ \mathcal{E}_{\tau_{n+1}, n+1} &= \mathrm{Retrieve}(\boldsymbol{\tau}_{n+1}; \mathcal{B}_{n+1}), \end{aligned} \tag{23}$$

其中，$\mathrm{m}_n = (r_n, \bar{\ell}_n, \bar{\kappa}_n)$ 包含奖励、平均时延和平均货币成本。按照阶段 III 的保留规则，当已完成的支架未带来可接受的改进时，$\mathrm{Update}_{\mathrm{III}}$ 保持 $\mathcal{B}_n$ 不变。否则，被保留的支架将成为后续任务的潜在参考。因此，流式推理通过持续演化的支架库迁移既有经验，既不会将环境反馈注入当前轨迹采样，也不会更新模型参数。

6 实验与分析

6.1 实验设置

评测基准。 我们在九个基准上评测 JIT-Agent ，并将其归为四类任务：

- **深度研究：** BrowseComp-Plus (Chen 等, 2025b)（准确率）、DeepSearchQA (Gupta 等, 2026)（答案 F1）以及 xBench-DeepSearch（xBench-DS） (Chen 等, 2025a)（准确率）。
- **日常工作：** AgentIF-Oneday (Chen 等, 2026a)（归一化加权评分准则得分）和 PinchBench (PinchBench, 2026)（平均得分）。
- **规划：**来自 (Zhang 等, 2026d) 的 DeepPlanning-Shopping（购物车匹配率）与 DeepPlanning-Travel（综合约束得分）子集。
- **工作空间：** OfficeBench (Wang 等, 2024) 和 OdysseyBench (Wang 等, 2025b)（二者均采用任务成功率）。

这些基准共同覆盖证据密集型搜索、长时程指令遵循、约束感知规划以及多应用工作空间执行。所有指标均按 0–100 的尺度报告；数值越高越好。

骨干模型。 JIT-Agent 基于 Qwen3.6-27B 进行训练。我们主要以 GLM-5.2 (Z.ai, 2026) 和 DeepSeek-V4-Flash-Preview (DeepSeek-AI, 2026) 实例化 JIT-Agent 。这些骨干模型提供了互补的运行点：GLM-5.2 是一个强大的开源长时程模型，而 DeepSeek-V4-Flash 强调推理效率。为检验泛化性，我们还使用了 Qwen3.6、Mimo-V2.5 系列中的其他模型。

基线。 我们采用两组互补的基线。首先，广泛比较包括原始的 **Qwen3.7-Plus** (Qwen Team, 2026)、**GLM-5.2**、**DeepSeek-V4-Flash/Pro**（二者均为预览版本）、**Kimi K2.7 Code** (Moonshot AI, 2026)、**GPT-5.6** (OpenAI, 2026) 以及 **Gemini 3.1 Pro/3.5 Flash** (Google DeepMind, 2026a,b)。其次，在固定骨干模型的条件下，我们将 JIT-Agent 与五种可复用的智能体支架进行比较：Claude Code (Anthropics, 2025)、Codex (OpenAI, 2025)、OpenCode (OpenCode-AI, 2025)、Hermes (Nous Research, 2026) 和 NanoBot (HKUDS, 2026)。第一组衡量端到端竞争力，第二组则将支架设计的贡献与模型权重的贡献分离开来。

6.2 主要结果

表 3 将 JIT 生成的支架与原始智能体骨干模型及前沿模型基线进行比较。我们报告三个深度研究基准、两个日常工作基准、两个规划基准和两个工作空间基准的结果。

表 3: 九个智能体基准上的主要结果。以 JIT-Agent 为前缀的行与相应的原始版本使用同一骨干模型，但将其默认脚手架替换为 JIT 生成的支架。得分按 0–100 的尺度报告；数值越高越好。每列的第 1、第 2 和第 3 名结果分别以柔和绿色、黄绿色和黄色突出显示；第 1 名还以粗体标示。缺失的评测以 "—" 表示。

Model	Deep Research			Daily Work		Planning		Workspace	
	BC+	DSQA	xBench	AgentIF	PinchBench	Shop	Travel	Office	Odyssey
Qwen3.7-Plus	70.5	78.0	75.0	59.9	80.5	75.2	52.1	56.6	67.7
GLM-5.2	72.0	89.2	76.0	63.0	87.0	78.2	62.8	63.0	75.3
DeepSeek-V4-Flash	68.1	76.2	70.1	58.4	81.7	59.1	54.8	61.0	71.0
DeepSeek-V4-Pro	71.4	72.4	79.0	56.5	61.1	71.1	55.2	62.0	72.0
Kimi K2.7 Code	73.6	87.8	79.0	57.3	76.1	72.8	56.9	65.8	69.9
GPT-5.6	76.9	76.0	81.0	68.0	84.2	83.7	**84.9**	65.3	68.7
Gemini 3.1 Pro	67.5	75.8	83.0	60.1	81.0	77.4	70.7	60.6	74.0
Gemini 3.5 Flash	75.0	88.0	85.0	64.0	74.2	76.2	50.3	63.3	78.0
JIT-Agent + GLM-5.2	**78.0**	**93.9**	**88.0**	**69.9**	**93.3**	83.4	83.0	**68.4**	**78.7**
JIT-Agent + DeepSeek-V4-Flash	74.0	85.1	82.0	63.8	92.9	**83.9**	61.3	63.4	73.0

相同骨干模型上的一致增益。 在全部 18 个直接匹配的骨干模型–基准组合上，将默认脚手架替换为 JIT 生成的支架均能提升性能。在 GLM-5.2 上，九个基准的平均分从 74.1 提升至 81.8（+7.7 分）；在 DeepSeek-V4-Flash 上，则从 66.7 提升至 75.5（+8.8 分）。最大增益出现在需要持续状态管理和约束跟踪的任务上：DeepSeek-V4-Flash 在 DeepPlanning-Shopping 上提升 24.8 分（59.1 → 83.9），而 GLM-5.2 在 DeepPlanning-Travel 上提升 20.2 分（62.8 → 83.0）。搜索与开放式交互也获得显著收益，包括 DeepSeek-V4-Flash 在 xBench-DS 上的 +11.9 分以及在 DeepSearchQA 上的 +8.9 分。这些广泛增益表明，支架适配所改变的不只是提示风格：它还改善了骨干模型分配上下文、分解长时程目标和协调外部动作的方式。

与前沿模型的竞争力。 尽管采用开源骨干模型，配备 JIT 支架的系统仍在九个基准列中的八个取得最佳结果。具体而言，JIT-Agent + GLM-5.2 在七个基准上排名第一，在 DeepSearchQA、AgentIF 和 PinchBench 上分别达到 93.9、69.9 和 93.3；JIT-Agent + DeepSeek-V4-Flash 则以 83.9 在 DeepPlanning-Shopping 上领先。后一个组合在所有已报告基准上也都超过更强的 DeepSeek-V4-Pro 基线，平均优势为 8.7 分。DeepPlanning-Travel 是唯一未由 JIT 支架模型领先的列，其中 GLM-5.2 + JIT-Agent 达到 83.0，与 GPT-5.6 仅相差 1.9 分。总体而言，这些结果支持本文的核心主张：改进运行脚手架可以获得原本需通过扩展骨干模型才能追求到的相当一部分能力。

6.3 与先进智能体支架的比较

除与原始骨干模型比较外，我们还控制底层模型，仅改变支架。我们分别在 DeepSeek-V4-Flash 和 Qwen3.6-Flash 上实例化各支架，并在 DeepSearchQA、xBench-DS 和 AgentIF 上进行评测。除任务性能外，我们还报告每个样例的平均词元消耗和 API 成本，以区分有效编排带来的收益与仅靠投入更多推理计算获得的收益。

控制骨干模型时的性能。 JIT-Agent 在六个骨干模型–基准设置中的四个取得最高性能。使用 DeepSeek-V4-Flash 时，它在 DeepSearchQA 上比最强固定支架高 4.7 分（85.1 对 80.4），在 xBench-DS 上高 4.0 分（82.0 对 78.0）。使用 Qwen3.6-Flash 时，它在 xBench-DS 上获得 7.0 分增益（70.0 对 63.0），在 AgentIF 上获得 2.9 分增益（58.3 对 55.4）。两个例外呈现出有限的质量–效率权衡：在 DeepSeek-V4-Flash 的 AgentIF 上，JIT-Agent 落后 Claude Code 3.1 分；在 Qwen3.6-Flash 的 DeepSearchQA 上，它落后 NanoBot 3.9 分，但两种情况下使用的词元都显著更少。没有任何固定支架能在所有任务上占据优势：例如，NanoBot 在 Qwen3.6-Flash 的 DeepSearchQA 上最强，但在 AgentIF 上落后 JIT-Agent 14.8 分。这种差异表明，应生成任务条件化的支架，而非在全局选择一种长期固定的脚手架。

表 4: 与先进智能体支架的受控比较。对于每个固定的骨干模型，我们报告任务性能（Perf.）、以千为单位的每样例平均词元消耗（#Tokens (K)）以及以美元计的每样例 API 成本。性能越高越好；词元数和成本越低越好。每个骨干模型组内的最佳值以粗体标示。

Backbone	Harness	DeepSearchQA			xBench-DS			AgentIF		
		Perf. ↑	#Tokens (K)	Cost ↓	Perf. ↑	#Tokens (K)	Cost ↓	Perf. ↑	#Tokens (K)	Cost ↓
DeepSeek-V4-Flash	Claude Code	79.6	625	$0.088	75.0	559	$0.079	**66.9**	808	$0.114
	Codex	77.8	760	$0.107	70.0	680	$0.096	58.5	870	$0.123
	OpenCode	75.9	1,832	$0.258	65.0	1,157	$0.159	48.1	950	$0.135
	Hermes	69.9	1,157	$0.163	72.0	1,254	$0.177	60.3	1,000	$0.142
	NanoBot	80.4	924	$0.131	78.0	527	$0.075	53.1	1,034	$0.147
	JIT-Agent	**85.1**	**400**	**$0.066**	**82.0**	**212**	**$0.039**	63.8	**476**	**$0.097**
Qwen3.6-Flash	Claude Code	72.8	710	$0.140	58.0	650	$0.128	55.4	900	$0.177
	Codex	68.5	980	$0.193	52.0	874	$0.172	34.2	839	$0.170
	OpenCode	64.1	1,968	$0.384	44.0	1,300	$0.256	38.7	961	$0.217
	Hermes	60.8	1,319	$0.256	36.0	1,155	$0.227	49.7	1,184	$0.223
	NanoBot	**74.2**	892	$0.197	63.0	597	$0.119	43.5	950	$0.187
	JIT-Agent	70.3	**464**	**$0.095**	**70.0**	**300**	**$0.069**	**58.3**	**394**	**$0.078**

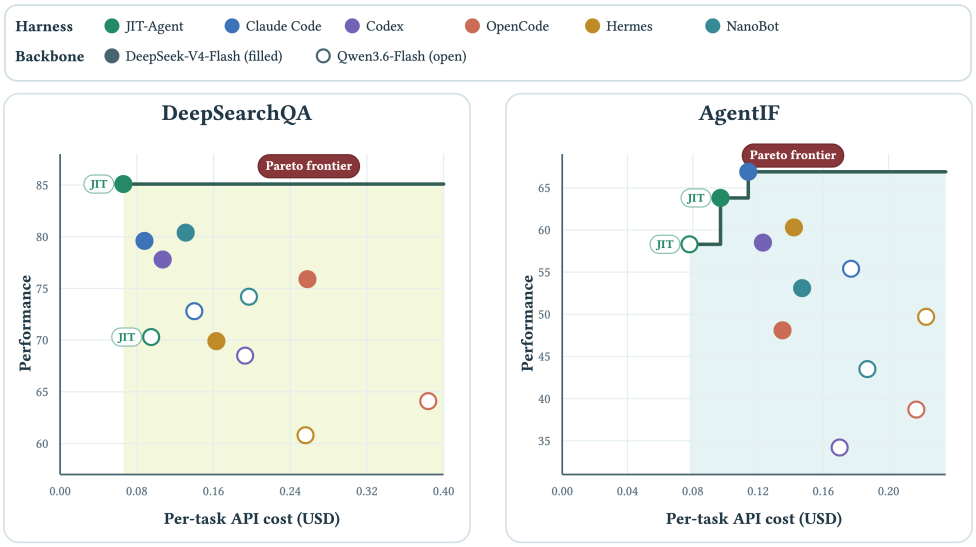


图 4: **DeepSearchQA 和 AgentIF 上的成本–性能权衡**。标记颜色表示支架，实心圆和空心圆则区分 DeepSeek-V4-Flash 与 Qwen3.6-Flash。横轴为每样例的 API 成本（美元），纵轴为任务性能。深绿色阶梯线描绘全局帕累托前沿，浅黄绿色和蓝绿色区域包含至少被一个帕累托最优点支配的组合。

词元与成本效率。 JIT-Agent 在全部六个受控设置中都具有最低的词元消耗和 API 成本。与每种设置中最便宜的固定支架相比，它将每样例成本降低了 14.9–54.1%，平均降幅为 36.0%。在 DeepSeek-V4-Flash 的 xBench-DS 上，词元消耗从 527K 降至 212K，成本从 $0.075 降至 $0.039，而性能则从 78.0 提升至 82.0。最大成本降幅出现在 Qwen3.6-Flash 的 AgentIF 上：JIT-Agent 使用 394K 词元，成本为 $0.078；相比之下，固定替代方案至少使用 839K 词元，成本为 $0.170，但前者仍将固定支架的最佳得分从 55.4 提升至 58.3。因此，这些增益无法由更长的轨迹解释；生成的支架通常通过更短、更有选择性的执行取得更强结果。

6.4 成本–性能帕累托前沿

图 4 展示了 DeepSearchQA 和 AgentIF 受控比较中的成本–性能几何关系。每个点都是表 4 中的一个骨干模型–支架组合；越靠近左上角的点越理想。在 DeepSearchQA 上，DeepSeek-V4-Flash + JIT-Agent 是已评测组合中的一个强帕累托最优运行点，取得 85.1 分，每样例成本为 $0.066。相较最强固定支架 NanoBot，它提升 4.7 分，同时将成本降低 49.6%（$0.131 → $0.066）。对于

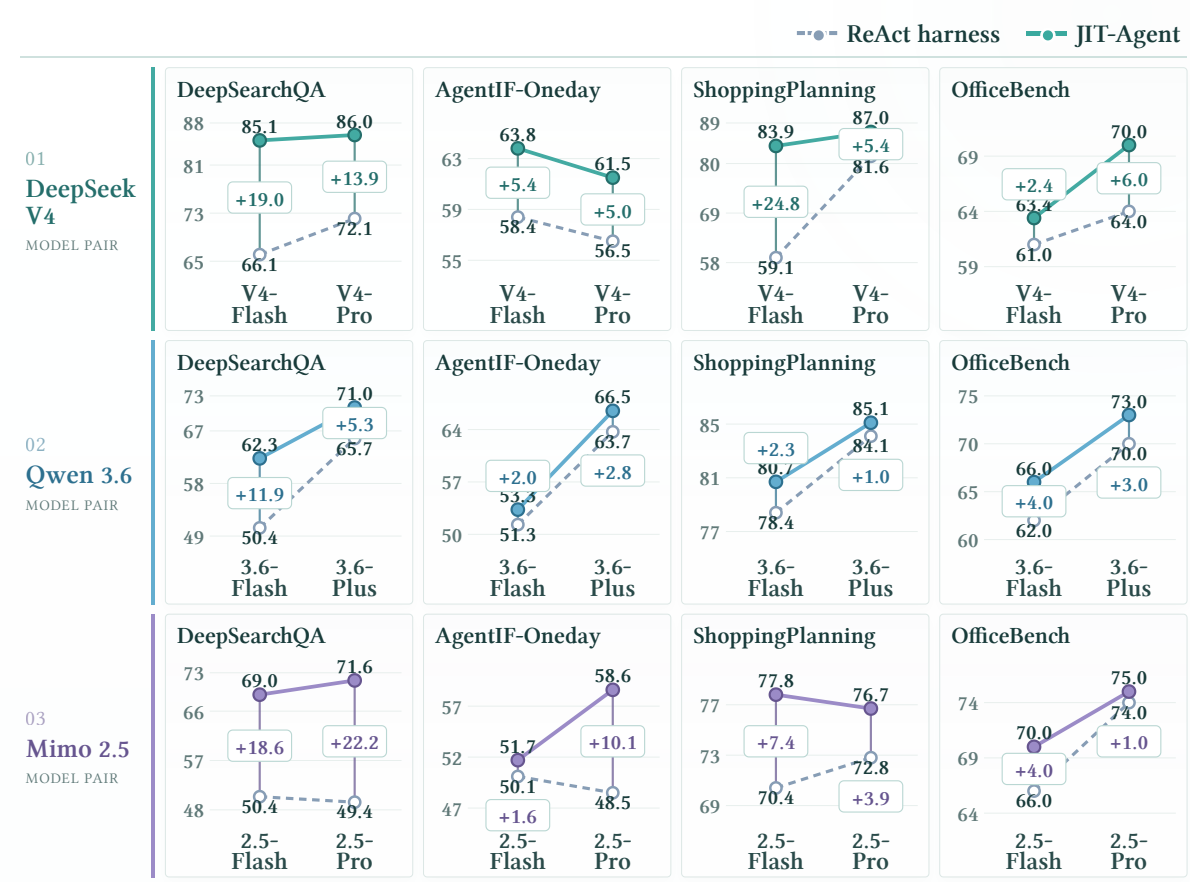


图 5: JIT 生成的支架相较 ReAct 持续提升配对骨干模型的性能。各行对应三个模型家族及每个家族中的两个变体；各列对应 DeepSearchQA、AgentIF-Oneday、DeepPlanning-Shopping 和 OfficeBench。每个面板都比较同一骨干模型在固定 ReAct 支架（虚线）与 JIT 生成支架（实线）下的表现，标注框报告绝对得分增益。DeepSearchQA 使用包含 100 个样例的子集；其余三个基准使用包含 50 个样例的子集。

Qwen3.6-Flash，生成式支架提供了一个计算量更低的运行点：它取得 70.3 分，成本为 $0.095，而 NanoBot 取得 74.2 分，成本为 $0.197，即削减 51.8% 的成本，但需权衡 3.9 分。AgentIF 呈现出不同但互补的前沿。使用 DeepSeek-V4-Flash 时，JIT-Agent 相较 Claude Code 将成本降低 14.9%（$0.097 对 $0.114），同时性能差距保持在 3.1 分以内（63.8 对 66.9）；因此二者都位于前沿上。使用 Qwen3.6-Flash 时，JIT-Agent 严格支配所有固定支架，将最佳得分从 55.4 提升至 58.3，同时把观测到的最低成本从 $0.170 降至 $0.078。因此，该前沿取决于任务，但 JIT 点一致向左移动，说明任务自适应支架提升了推理效率，而不是通过更长轨迹来换取准确率。

6.5 跨模型对的泛化

为考察 JIT 支架生成能否跨模型家族及变体迁移，我们评测了涵盖三个模型对的六个骨干模型：DeepSeek-V4-Flash/Pro、Qwen3.6-Flash/Plus 和 Mimo-V2.5-Flash/Pro。对于每个骨干模型，我们固定模型，并在 DeepSearchQA、AgentIF-Oneday、DeepPlanning-Shopping 和 OfficeBench 上比较其标准 ReAct 支架与 JIT 生成的支架。

在全部 24 个直接匹配的骨干模型–基准比较中，JIT 生成的支架均优于 ReAct，平均增益为 7.6 分。这一效果在每个家族以及每个模型对的两个变体上都成立：DeepSeek V4、Qwen 3.6 和 Mimo 2.5 的平均提升分别为 10.2、4.0 和 8.6 分。DeepSearchQA 的平均增益最大，达到 15.2 分，其中 Mimo-V2.5-Pro 提升 +22.2 分，DeepSeek-V4-Flash 提升 +19.0 分；DeepPlanning-Shopping 平均提升 7.5 分，其中 DeepSeek-V4-Flash 的增益为 +24.8 分。这些在同一骨干模型内的一致增益表明，支架智能能够跨模型家族和变体迁移，而非仅用于弥补某个特定骨干模型的不足。

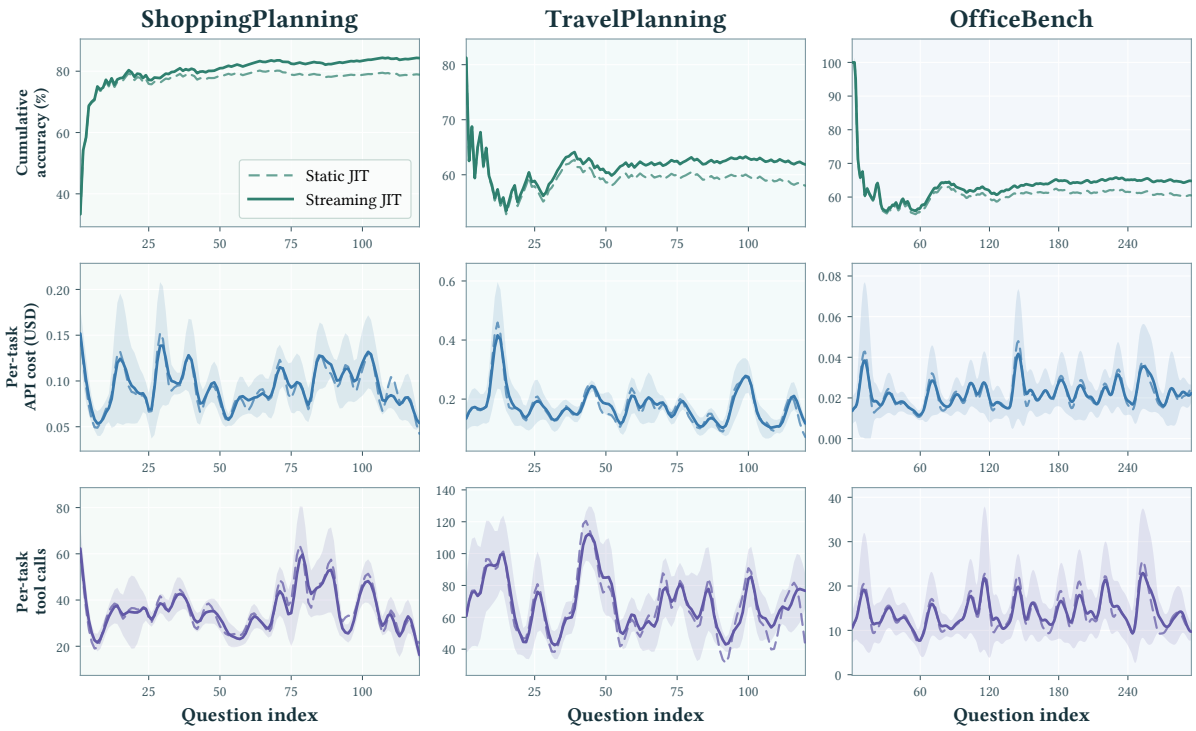


图 6: 跨任务流的流式测试时支架演化。 DeepPlanning-Shopping、DeepPlanning-Travel 和 OfficeBench 上的累积准确率（上）、每任务 API 成本（中）和每任务工具调用次数（下）。虚线表示静态 JIT，其中各任务专属支架彼此独立生成；实线表示流式 JIT，随着新任务到来持续纳入执行反馈。阴影区域表示流式轨迹周围的局部变化。流式 JIT 在全部三个基准上均以更高的累积准确率结束，而 API 成本和工具使用轨迹仍取决于任务，且规模大体相近。

6.6 测试时支架演化

JIT-Agent 的设计目标不仅是一次性生成支架，还要利用执行反馈改进其档案库。我们比较了支架生成彼此独立的静态 *JIT* 与在整个评测流中检索和更新支架的流式 *JIT*。

在 DeepPlanning-Shopping、DeepPlanning-Travel 和 OfficeBench 上，流式 JIT 的最终表现均高于静态变体。在三个任务流中，随着执行反馈不断积累，累积准确率优势逐渐显现，并一直保持到评测结束。相应的成本和工具调用轨迹表明，终点增益并不总是与更大的交互预算相伴。这一模式与 Evo-GDPO 的目标一致：仅当支架能够推进档案库前沿时才予以保留。

6.7 生成支架的可视化

定量结果证实生成式支架能够改善执行；接下来我们考察 JIT-Agent 实际生成的内容。图 7 和 8 可视化了在同一四模块协议下，为计算结构截然不同的任务生成的两个支架。二者的差异不止体现在提示或模块名称上：生成的规划表示、动作拓扑、持久状态和开放能力都会围绕各任务的需求重新组织。

将产物生产表示为依赖图。 Palimpsest 是为一个跨应用任务生成的：该任务必须发现联系人卡片、排除一人、规范化并排序其余记录、创建工作簿，最后通过电子邮件发送。支架将这些要求编译成显式 DAG：发现与模式检查为筛选提供前提，工作簿构建等待规范化记录，而交付则等待产物验证。随后，`GraphPlanAction` 以有界宽度和深度执行已就绪节点；同时，`GraphPlanMemory` 为中间目标、证据和产物建立索引，使下游节点直接使用已完成的结果，而不是从交互记录中重新构建。所得支架与任务的产物依赖关系及提交顺序相匹配，将冗长的自然语言请求转化为可验证的生产流水线。

将深度研究实现为有界递归委派。 Trapdoor 处理一个多跳身份问题，其线索横跨一段历史引文、一次大学合并、一本回忆录和一篇研究论文。此处，固定 DAG 会过早选定一条尚不确定的证据路

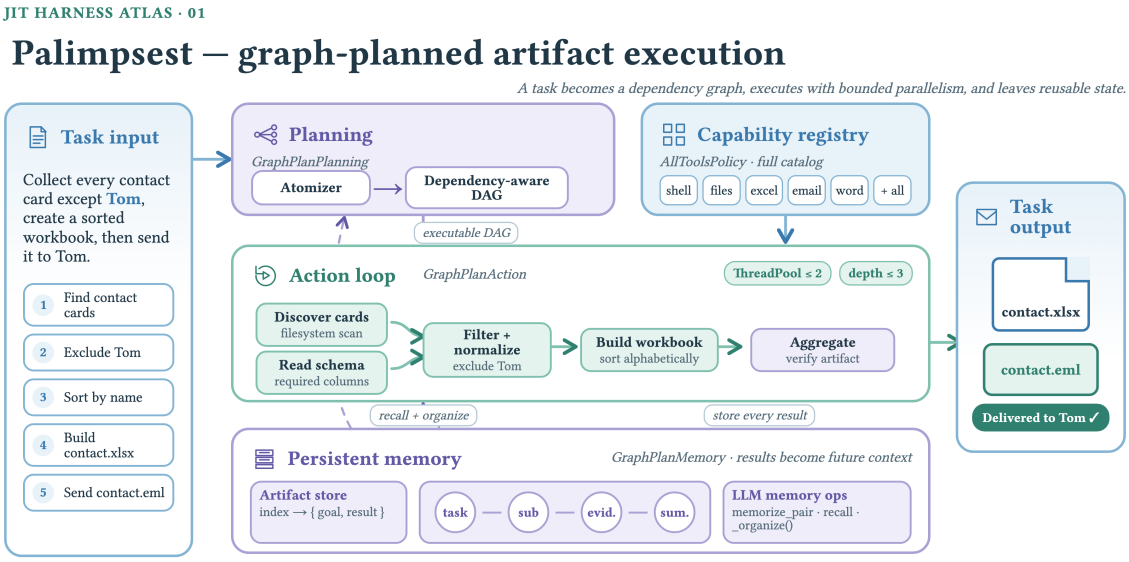


图 7: **Palimpsest**：以图规划的产物执行。`GraphPlanPlanning` 将联系人处理请求转换为 DAG；`GraphPlanAction` 以有界宽度和深度执行该图，而 `GraphPlanMemory` 存储可复用产物与推理状态。

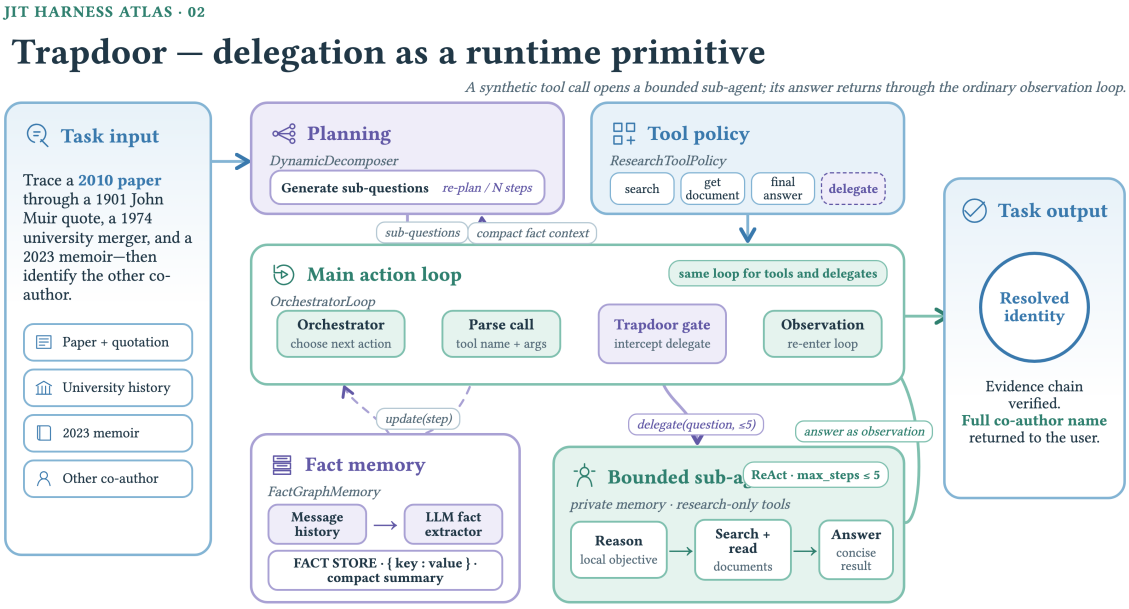


图 8: **Trapdoor**：委派工具调用背后的有界研究。合成的 `delegate` 能力由 `OrchestratorLoop` 截获，后者运行一个有界子智能体，并将提取的事实写入 `FactGraphMemory`。

径，因此生成的支架使用 `DynamicDecomposer` 构造并修订子问题，并以合成的 `delegate` 能力增强研究工具策略。一次 `delegate` 调用会启动一个私有研究子智能体，它拥有独立记忆、仅用于研究的工具以及五步预算；返回的答案作为普通观测重新进入父循环。与此同时，`FactGraphMemory` 从不断增长的历史中提取紧凑的键–值事实，使父智能体能够跨分支整合证据，而无需继承每条子智能体轨迹。因此，支架将递归研究变成一种运行时原语，专门用于通过分支证据收集来消除不确定性的任务。

这种对比是核心定性结果：同一个生成器将一个任务映射为带产物存储的图执行，将另一个任务映射为带事实存储的递归编排。共享协议约束的是接口，而非行为。节 A 中的其他案例将这一模式扩展到分层上下文折叠、以完成状态为门控的工具访问、阶段条件化执行、证据矩阵搜索、选择性上

下文渲染、类型化计算状态、确定性的逐文件验证以及局部动作修复；其流程见图 A.1 至 A.8。

7 结论与未来工作

结论。 我们提出了 JIT-Agent ，这是一个专门用于在推理时合成、修复和演化任务条件化智能体支架的模型。通过采用可组合的四模块协议表示支架，并通过定制学习、修复监督和 Evo-GDPO 训练生成器，JIT-Agent 将支架构建从人工设计的预先产物转变为一种习得能力。在深度研究、日常工作、规划和工作空间任务上，JIT 生成的支架持续增强其底层骨干模型，与前沿模型和先进固定支架竞争，并改善成本–性能权衡。这些结果确立了支架智能，即适配模型赖以行动的运行脚手架之能力，是一种超越单纯模型权重、可训练且可迁移的智能体能力来源。

未来工作。 我们将支架智能视为与模型容量和推理计算并列的另一个基础扩展维度，而 JIT-Agent 是迈向实现这一维度的一步。更长期的机遇在于一种模型–支架协同设计范式，其中基础模型与塑造其记忆、规划、动作和工具使用的运行结构共同训练。不过，我们认为未来系统不必遵循本文刻意探索的激进形式，即在即时阶段重新设计整个脚手架：生产运行时可以保留稳定核心，同时允许模型在任务需要时构建、修订或替换选定的支架组件。随着这一范式逐渐成熟，本文开发的支架合成、修复与演化能力将越来越多地被基础模型自身内化。届时，模型不仅会学习在给定支架中行动，还会学习改进其赖以行动的支架，从而围绕自适应接口、可验证的运行时修改以及模型与执行系统的联合扩展，开启广阔的研究议程。

贡献者

项目负责人 Guibin Zhang

核心贡献者 Guibin Zhang, Leo Lu, Fangzhou Xie

贡献者 Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, Yue Liao, Heng Chang, Xiaobin Hu, Qibing Ren

通讯作者 Wangchunshu Zhou, Shuicheng Yan

参考文献

Salaheddin Alzu'bi, Baran Nama, Arda Kaz, Anushri Eswaran, Weiyuan Chen, Sarvesh Khetan, Rishab Bala, Tu Vu, 和 Sewoong Oh. Roma：面向长时程多智能体系统的递归开放元智能体框架, 2026. 链接 https://arxiv.org/abs/2602.01848.

Anthropics. GitHub - anthropics/claude-code：Claude Code 是一种运行于终端中的智能体编程工具，能够理解代码库，并通过自然语言命令执行常规任务、解释复杂代码和处理 git 工作流，帮助你更快编程。— github.com. https://github.com/anthropics/claude-code, 2025. [访问于 04-01-2026].

Apodex Team. Apodex-1.0：自演化的重型求解器. https://www.apodex.com/blog/apodex-1.0, 2026.

Fei Bai, Huatong Song, Shuang Sun, Daixuan Cheng, Yike Yang, Chuan Hao, Renyuan Li, Feng Chang, Yuan Wei, Ran Tao, Bryan Dai, Jian Yang, Wayne Xin Zhao, 和 Ji-Rong Wen. Clawgym：构建高效 claw 智能体的可扩展框架, 2026. 链接 https://arxiv.org/abs/2604.26904.

James Burgess, Jan N. Hansen, Duo Peng, Yuhui Zhang, Alejandro Lozano, Min Woo Sun, Emma Lundberg, 和 Serena Yeung-Levy. Papersearchqa：借助 rlvr 学习在科学论文中搜索与推理, 2026. 链接 https://arxiv.org/abs/2601.18207.

Pengfei Cao, Tianyi Men, Wencan Liu, Jingwen Zhang, Xuzhao Li, Xixun Lin, Dianbo Sui, Yanan Cao, Kang Liu, 和 Jun Zhao. 用于规划的大语言模型：全面而系统的综述, 2025. 链接 https://arxiv.org/abs/2505.19683.

Angelica Chen, David Dohan, 和 David So. Evoprompting：用于代码级神经架构搜索的语言模型. 神经信息处理系统进展, 36:7787–7817, 2023.

Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, Haotong Tian, Tianbao Xie, Fangfu Liu, Haoye Zhang, Hongzhang Liu, Yuan Gong, Chen Sun, Han Hou, Hui Yang, James Pan, Jianan Lou, Jiayi Mao, Jizheng Liu, Jinpeng Li, Kangyi Liu, Kenkun Liu, Rui Wang, Run Li, Tong Niu, Wenlong Zhang, Wenqi Yan, Xuanzheng Wang, Yuchen Zhang, Yi-Hsin Hung, Yuan Jiang, Zexuan Liu, Zihan Yin, Zijian Ma, 和 Zhiwen Mo. xbench：通过与职业对齐的真实世界评测跟踪智能体生产力扩展, 2025a. 链接 https://arxiv.org/abs/2506.13651.

Kaiyuan Chen, Qimin Wu, Taiyu Hou, Tianhao Tang, Xueyu Hu, Yuchen Hou, Bikun Li, Chengming Qian, Guoyin Wang, Haolin Chen, Haotong Tian, Haoye Zhang, Haoyu Bian, Hongbing Pan, Hongkang Zhang, Hongyi Zhou, Jiaqi Cai, Jiewu Rao, Jiyuan Ren, Keduan Huang, Lucia Zhu Huang, Mingyu Yuan, Naixu Guo, Qicheng Tang, Qinyan Zhang, Shuai Chen, Siheng Chen, Ting Ting Li, Xiaoxing Guo, Yaocheng Zuo, Yaoqi Guo, Yinan Wang, Yinzhou Yu, Yize Wang, Yuan Jiang, Yuan Tian, Yuanshuo Zhang, Yuxuan Liu, Yvette Yan Zeng, Zenyu Shan, Zihan Yin, Xiaobo Hu, Yang Liu, Yixin Ren, 和 Yuan Gong. Agentif-oneday：面向日常场景通用 ai 智能体的任务级指令遵循基准, 2026a. 链接 https://arxiv.org/abs/2601.20613.

Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, 和 Jian Luan. Harnessx：可组合、自适应且可演化的智能体支架工厂, 2026b. 链接 https://arxiv.org/abs/2606.14249.

Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhu Chen, 和 Jimmy Lin. Browsecomp-plus：更公平、更透明的深度研究智能体评测基准, 2025b. 链接 https://arxiv.org/abs/2508.06600.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, 和 Deshraj Yadav. Mem0：以可扩展长期记忆构建生产就绪的 ai 智能体, 2025. 链接 https://arxiv.org/abs/2504.19413.

ClaudeCode. 智能体技能 - Claude Code 文档 — code.claude.com. https://code.claude.com/docs/en/skills, 2025. [访问于 03-01-2026].

DeepSeek-AI. Deepseek-v4 预览版发布. https://api-docs.deepseek.com/news/news260424/, 2026.

Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, 和 Amir Gholami. Plan-and-act：改进智能体对长时程任务的规划. *arXiv* 预印本 *arXiv:2503.09572*, 2025a.

Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, 和 Amir Gholami. Plan-and-act：改进智能体对长时程任务的规划, 2025b. 链接 https://arxiv.org/abs/2503.09572.

Google DeepMind. Gemini 3.1 pro 模型卡. https://deepmind.google/models/model-cards/gemini-3-1-pro, 2026a.

Google DeepMind. Gemini 3.5 flash 模型卡. https://deepmind.google/models/model-cards/gemini-3-5-flash/, 2026b.

Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, 和 Dipanjan Das. Deepsearchqa：弥合深度研究智能体的全面性差距, 2026. 链接 https://arxiv.org/abs/2601.20975.

HKUDS. Nanobot：轻量级开源 ai 智能体. https://github.com/HKUDS/nanobot, 2026.

Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, 和 Yafeng Deng. Evermemos：面向结构化长时程推理的自组织记忆操作系统, 2026a. 链接 https://arxiv.org/abs/2601.02163.

Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, 和 Ping Luo. Hiagent：用于以大语言模型解决长时程智能体任务的分层工作记忆管理, 2024a. 链接 https://arxiv.org/abs/2408.09559.

Shengran Hu, Cong Lu, 和 Jeff Clune. 智能体系统的自动化设计. *arXiv* 预印本 *arXiv:2408.08435*, 2024b.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, 和 Shuicheng Yan. Ai 智能体时代的记忆, 2025. 链接 https://arxiv.org/abs/2512.13564.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, 和 Shuicheng Yan. Ai 智能体时代的记忆, 2026b. 链接 https://arxiv.org/abs/2512.13564.

Jiajie Jin, Xiaoxi Li, Guanting Dong, Yuyao Zhang, Yutao Zhu, Yang Zhao, Hongjin Qian, 和 Zhicheng Dou. Hira：用于深度搜索中解耦规划与执行的分层推理框架, 2025. 链接 https://arxiv.org/abs/2507.02652.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, 等. Dspy：将声明式语言模型调用编译为自我改进流水线. *arXiv* 预印本 *arXiv:2310.03714*, 2023.

LangGraph. GitHub - langchain-ai/langgraph：以图的形式构建稳健的语言智能体。— github.com. https://github.com/langchain-ai/langgraph, 2025. [访问于 03-01-2026].

Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, 和 Yujin Tang. 递归式支架自我改进, 2026a. 链接 https://arxiv.org/abs/2607.15524.

Sanghoon Lee, Jiyeong Chae, 和 Kyung-Joon Park. 面向物理 ai 的支架工程：机器人中间件即支架层, 2026b. 链接 https://arxiv.org/abs/2606.09416.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, 和 Chelsea Finn. Meta-harness：模型支架的端到端优化, 2026c. 链接 https://arxiv.org/abs/2603.28052.

Yoonsang Lee, Howard Yen, Xi Ye, 和 Danqi Chen. 用于并行扩展长时程智能体任务的智能体式聚合, 2026d. 链接 https://arxiv.org/abs/2604.11753.

Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, 和 Shuyue Hu. 在生态系统规模上组织、编排和评测智能体技能, 2026a. 链接 https://arxiv.org/abs/2603.02176.

Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, 和 Zhicheng Dou. Deepagent：具备可扩展工具集的通用推理智能体, 2026b. 链接 https://arxiv.org/abs/2510.21618.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, 和 Yu-Gang Jiang. 智能体支架工程：可观测性驱动的编程智能体支架自动演化, 2026. 链接 https://arxiv.org/abs/2604.25850.

Jiaxi Liu, Yanzuo Jiang, Guibin Zhang, Zihan Zhang, Heng Chang, Zhenfei Yin, Qibing Ren, 和 Junchi Yan. TodoEvolve：学习构建智能体规划系统, 2026a. 链接 https://arxiv.org/abs/2602.07839.

Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, 和 Pavlo Molchanov. GDPO：面向多奖励强化学习优化的组奖励解耦归一化策略优化, 2026b. 链接 https://arxiv.org/abs/2601.05242.

Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, 和 Hanqing Lu. 自适应自动支架：面向开放式任务流智能体系统部署的持续自我改进, 2026c. 链接 https://arxiv.org/abs/2606.01770.

Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, 和 Kevin P. Murphy. AutoHarness：通过自动合成代码支架改进 LLM 智能体, 2026. 链接 https://arxiv.org/abs/2603.03329.

Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, 和 Ming Zhang. 大语言模型智能体：方法、应用与挑战综述, 2025. 链接 https://arxiv.org/abs/2503.21460.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, 和 Peter Clark. Self-Refine：利用自我反馈进行迭代优化, 2023.

Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, 和 Yao Hu. 面向大语言模型智能体的智能体支架：综述. 预印本, 四月 2026a. doi: 10.20944/preprints202604.0428.v2. 链接 https://doi.org/10.20944/preprints202604.0428.v2.

Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, 和 Yao Hu. 面向大语言模型智能体的智能体支架：综述. 预印本, 2026b.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue

Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, 和 Ludwig Schmidt. Terminal-Bench：在命令行界面的高难度真实任务上评测智能体, 2026. 链接 https://arxiv.org/abs/2601.11868.

Microsoft. GitHub - ag2ai/ag2：AG2（前称 AutoGen）：开源智能体操作系统. https://github.com/ag2ai/ag2, 2024. [访问于 31-08-2025].

Moonshot AI. Kimi K2.7 Code：开源智能体编程模型. https://www.kimi.com/resources/kimi-k2-7-code, 2026.

Jun Nie, Yonggang Zhang, Qianshu Cai, Yiu-ming Cheung, Xinmei Tian, 和 Bo Han. EvolveNet：面向智能体自我改进的协同支架演化, 2026a. 链接 https://arxiv.org/abs/2608.04968.

Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, 和 Bo Han. TTHE：测试时支架演化, 2026b. 链接 https://arxiv.org/abs/2607.08124.

Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, 等. 将代码作为智能体支架. *arXiv* 预印本 *arXiv:2605.18747*, 2026.

Nous Research. Hermes 智能体. https://github.com/NousResearch/hermes-agent, 2026.

OpenAI. Codex | 来自 OpenAI 的 AI 编程伙伴 — openai.com. https://openai.com/codex/, 2025.

OpenAI. GPT-5.6：随你的雄心共同扩展的前沿智能. https://openai.com/index/gpt-5-6/, 2026.

OpenClaw Contributors. OpenClaw：你的专属个人 AI 助手. https://github.com/openclaw/openclaw, 2026.

OpenCode-AI. GitHub - opencode-ai/opencode：强大的 AI 编程智能体，专为终端打造。— github.com. https://github.com/opencode-ai/opencode, 2025.

Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, 和 Hai-Tao Zheng. 自然语言智能体支架. *arXiv* 预印本 *arXiv:2603.25723*, 2026a.

Wenbo Pan, Shujie Liu, Chin-Yew Lin, Jingying Zeng, Xianfeng Tang, Xiangyang Zhou, Yan Lu, 和 Xiaohua Jia. 在黑暗中演化智能体：通过自偏好进行回顾式支架优化, 2026b. 链接 https://arxiv.org/abs/2606.05922.

PinchBench. 按成功率排名的最佳模型 | PinchBench — pinchbench.com. https://pinchbench.com/, 2026. [访问于 11-04-2026].

Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, 和 Heng Ji. ToolRL：奖励就是工具学习所需的一切. *arXiv* 预印本 *arXiv:2504.13958*, 2025.

Hongjin Qian, Zhao Cao, 和 Zheng Liu. MemoBrain：作为智能体推理大脑的执行记忆, 2026. 链接 https://arxiv.org/abs/2601.08079.

Tianrui Qin, Qianben Chen, Sinuo Wang, He Xing, King Zhu, He Zhu, Dingfeng Shi, Xinxin Liu, Ge Zhang, Jiaheng Liu, Yuchen Eleanor Jiang, Xitong Gao, 和 Wangchunshu Zhou. Flash-Searcher：通过基于 DAG 的并行执行构建快速而高效的网页智能体, 2025. 链接 https://arxiv.org/abs/2509.25301.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, 和 Maosong Sun. ToolLLM：帮助大语言模型掌握 16000 多个真实世界 API, 2023. 链接 https://arxiv.org/abs/2307.16789.

Qwen Team. Qwen3.7-Plus：多模态智能体智能. https://qwen.ai/home, 2026.

Jianhao Ruan, Zhihao Xu, Yiran Peng, Fashen Ren, Zhaoyang Yu, Xinbing Liang, Jinyu Xiang, Yongru Chen, Bang Liu, Chenglin Wu, Yuyu Luo, 和 Jiayi Zhang. AOrchestra：为智能体编排自动创建子智能体, 2026. 链接 https://arxiv.org/abs/2602.03786.

Anjana Sarkar 和 Soumyendu Sarkar. 采用 MCP 的 LLM 智能体通信综述：以软件设计模式为中心的回顾, 2025. 链接 https://arxiv.org/abs/2506.05364.

Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, 和 Yong Li. AgentSquare：在模块化设计空间中自动搜索 LLM 智能体. *arXiv* 预印本 *arXiv:2410.06153*, 2024.

Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, 和 Weinan Zhang. Harness-R1：从智能体失败轨迹中学习编辑可执行运行时支架, 2026. 链接 https://arxiv.org/abs/2608.02276.

Dingfeng Shi, Jingyi Cao, Qianben Chen, Weichen Sun, Weizhen Li, Hongxuan Lu, Fangchen Dong, Tianrui Qin, King Zhu, Minghao Liu, Jian Yang, Ge Zhang, Jiaheng Liu, Changwang Zhang, Jun Wang, Yuchen Eleanor Jiang, 和 Wangchunshu Zhou. Taskcraft：智能体任务的自动生成, 2025. 链接 https://arxiv.org/abs/2506.10055.

Yifan Shi, Wei Zhang, 和 Tianyi Cui. 面向时空可组合性的编程范式, 2026. 链接 https://github.com/cordiverse/paper. 预印本，2026 年 8 月 13 日草稿.

Noah Shinn, Beck Labash, 和 Ashwin Gopinath. Reflexion：具备动态记忆和自我反思的自主智能体. *arXiv* 预印本, abs/2303.11366, 2023. doi: 10.48550/arXiv.2303.11366. 链接 https://doi.org/10.48550/arXiv.2303.11366.

Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, 和 Zhicheng Dou. Envscaler：通过程序化合成扩展面向 llm 智能体的工具交互环境, 2026. 链接 https://arxiv.org/abs/2601.05808.

Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, 和 Chao Zhang. Adaplanner：语言模型基于反馈的自适应规划. 神经信息处理系统进展, 36:58202–58245, 2023.

Hongru Wang, Cheng Qian, Wanjun Zhong, Xiusi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, 和 Heng Ji. Otc：通过强化学习实现最优工具调用. *arXiv* 预印本 *arXiv:2504.14870*, 2025a.

Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, 和 Saravan Rajmohan. Odysseybench：在长时程复杂办公应用工作流上评测 llm 智能体, 2025b. 链接 https://arxiv.org/abs/2508.09124.

Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, 和 Jingbo Shang. Officebench：在用于办公自动化的多个应用中评测语言智能体, 2024. 链接 https://arxiv.org/abs/2407.19056.

Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, 和 Fei Huang. Webwalker：评测 llm 的网页遍历能力, 2025. 链接 https://arxiv.org/abs/2501.07572.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, 和 Chi Wang. Autogen：通过多智能体对话框架赋能下一代 llm 应用, 2023 年 8 月 1 日 2023.

Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, 和 Jingren Zhou. Resum：通过上下文摘要释放长时程搜索智能, 2026. 链接 https://arxiv.org/abs/2509.13313.

Renjun Xu 和 Yang Yan. 大语言模型的智能体技能：架构、获取、安全与未来路径, 2026. 链接 https://arxiv.org/abs/2602.12430.

B. Y. Yan, Chaofan Li, Hongjin Qian, Shuqi Lu, 和 Zheng Liu. 通过深度研究实现通用智能体记忆, 2025. 链接 https://arxiv.org/abs/2511.18423.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, 和 Ofir Press. Swe-agent：智能体–计算机接口赋能自动化软件工程, 2024. 链接 https://arxiv.org/abs/2405.15793.

Xiaofang Yang, Lijun Li, Heng Zhou, Tong Zhu, Xiaoye Qu, Yuchen Fan, Qianshan Wei, Rui Ye, Li Kang, Yiran Qin, Zhiqiang Kou, Daizong Liu, Qi Li, Ning Ding, Siheng Chen, 和 Jing Shao. 迈向高效智能体：记忆、工具学习与规划, 2026. 链接 https://arxiv.org/abs/2601.14192.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, 和 Yuan Cao. React：协同语言模型中的推理与行动, 2023a. 链接 https://arxiv.org/abs/2210.03629.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, 和 Yuan Cao. React：协同语言模型中的推理与行动. 载于第十一届学习表征国际会议, 2023b.

Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Siheng Chen, Jingren Zhou, 和 Yong Jiang. Agentfold：具备主动上下文管理的长时程网页智能体, 2025. 链接 https://arxiv.org/abs/2510.24699.

Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, 和 Deqing Yang. Evoagent：通过演化算法迈向多智能体自动生成. *arXiv* 预印本 *arXiv:2406.14228*, 2024.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, 和 James Zou. Textgrad：通过文本实现自动“微分”, 2024. 链接 https://arxiv.org/abs/2406.07496.

Z.ai. Glm-5.2：为长时程任务而构建. https://z.ai/blog/glm-5.2, 2026.

Alex L. Zhang, Tim Kraska, 和 Omar Khattab. 递归语言模型, 2026a. 链接 https://arxiv.org/abs/2512.24601.

Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, 和 Xiang Wang. 通过智能体超网络进行多智能体架构搜索. *arXiv* 预印本 *arXiv:2502.04180*, 2025a.

Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, 和 Shuicheng Yan. Memevolve：智能体记忆系统的元演化, 2025b. 链接 https://arxiv.org/abs/2512.18746.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, 和 Shuyue Hu. Self-harness：能够自我改进的支架, 2026b. 链接 https://arxiv.org/abs/2606.09498.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, 和 Chenglin Wu. AFlow：自动化生成智能体工作流, 十月 2024. 链接 http://arxiv.org/abs/2410.10762. arXiv:2410.10762.

Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, Yizhou Jin, 和 Yan Xu. HarnessCompass：引导支架自动演化，迈向可泛化且有效的智能体支架, 2026c. 链接 https://arxiv.org/abs/2608.01918.

Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, 和 Junyang Lin. Deepplanning：使用可验证约束评测长时程智能体规划, 2026d. 链接 https://arxiv.org/abs/2601.18137.

Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Zhengru Fang, Yi Nie, Changxu Wei, Yu Wang, Wenbo Ding, 和 Chao Yu. Harness vla：通过记忆引导的智能体将冻结 vla 导向可靠的操作原语, 2026e. 链接 https://arxiv.org/abs/2607.08448.

Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, 和 Jitao Sang. 将记忆作为动作：面向长时程智能体任务的自主上下文管理, 2025c. 链接 https://arxiv.org/abs/2510.12635.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, 和 Yu Su. Skillweaver：网页智能体可以通过发现和磨炼技能实现自我改进, 2025. 链接 https://arxiv.org/abs/2504.07079.

Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, 和 Weinan Zhang. Llm 智能体中的外部化：记忆、技能、协议与支架工程的统一综述, 2026. 链接 https://arxiv.org/abs/2604.08224.

Tailin Zhou. 分层自我改进：面向任务专属可演化智能体支架的框架, 2026. 链接 https://arxiv.org/abs/2608.08466.

He Zhu, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao, Hanhao Li, Ningning Wang, Pai Liu, Tianhao Peng, Xin Gui, Xiaowan Li, Yuhui Liu, Yuchen Eleanor Jiang, Jun Wang, Changwang Zhang, Xiangru Tang, Ge Zhang, Jian Yang, Minghao Liu, Xitong Gao, Jiaheng Liu, 和 Wangchunshu Zhou. Oagents：构建高效智能体的实证研究, 2025. 链接 https://arxiv.org/abs/2506.15741.

A 其他生成式支架可视化

本附录在节 6.7 的两个代表性案例之外，又补充了八个在同一四模块协议下生成的支架。这些示例涵盖约束繁多的购物与旅行、分阶段网页制作、线索驱动的研究、证据敏感型检索、数值分析以及工作空间操作。它们并非单一执行模板的变体：JIT-Agent 会选择适合每个请求结构的状态表示和控制机制，再围绕这一选择实例化规划、动作、记忆和能力编排。总体而言，这些案例从执行语义而非提示措辞层面直观展示了任务条件化的支架定制。

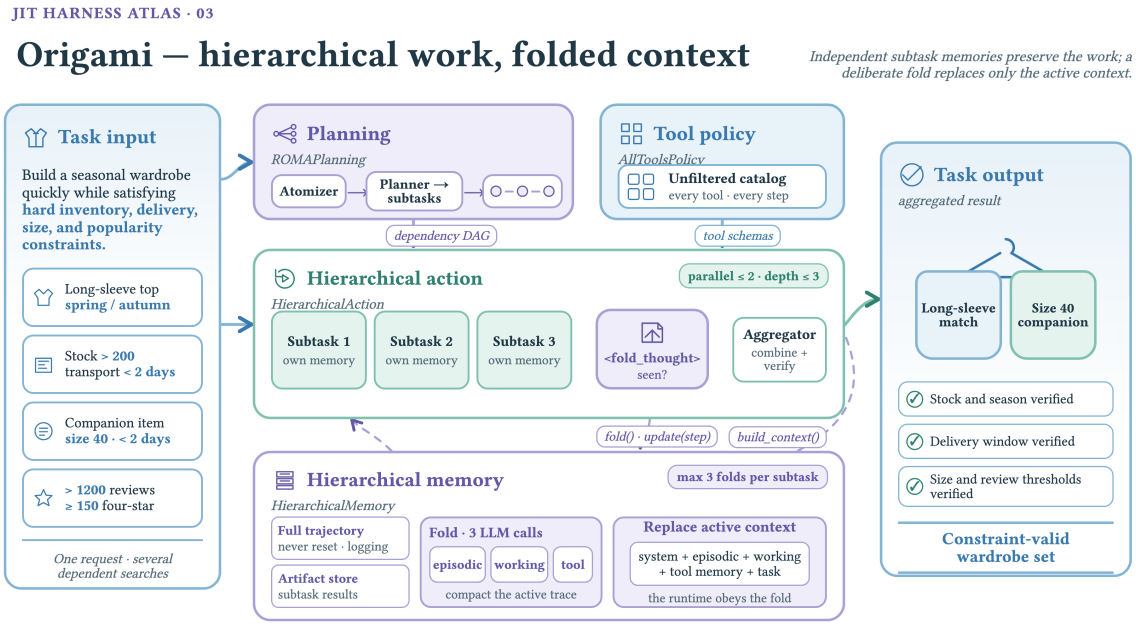


图 **A.1: Origami**：采用折叠上下文的分层工作。`ROMAPlanning` 创建相互隔离的子任务；`HierarchicalMemory` 保留其轨迹和产物，而 `fold_thought` 仅在聚合前替换当前工作上下文。

图 A.1 和 A.2 展示了两种通过记忆控制长时程执行的任务专属方式。ORIGAMI 必须在库存、配送、尺码、评论数量和评分等相互耦合的约束下搭配一套应季服装。`ROMAPlanning` 将请求分解为相互依赖的商品搜索，`HierarchicalAction` 以有界深度同时运行至多两个分支，每个分支都保留自己的轨迹和产物。当某个分支变长时，`fold_thought` 仅替换其当前工作上下文；已完成的子任务结果仍可用于最终约束检查。因此，该支架会将上下文用于组合式购物任务中尚未解决的部分，同时不丢弃先前的商品证据。

相比之下，TURNSTILE 服务于一个明确指定日期、乘客、房间、出发时间窗口、最高评分酒店、景点、用餐安排和每日路线的旅行请求。生成的 `TravelPlanning` 模块将这些义务编译为类型化的 `travel_spec` 和收集清单，`DataStoreMemory` 为每类证据分配专用存储桶。`DynamicToolPolicy` 为缺失的存储桶开放搜索工具，但在 `is_complete()` 成功前不会开放 `final_answer`。这里，记忆的主要用途并非压缩长轨迹；它充当一种可执行的覆盖契约，防止系统在收集完所需旅行事实前合成行程。

图 A.3 和 A.4 展示了显式状态如何持续重塑后续决策。GEARBOX 的生成目标是在指定驾车半径内寻找中餐馆，并将结果发布为交互式单页指南。其 `PhaseAction` 是共享的搜索–收集–构建寄存器的唯一写入者；只有收集到足够多的结构化餐馆记录后，才会转入构建阶段。`PhaseToolPolicy` 和 `PhaseAwareMemory` 都会读取该寄存器，从网页发现切换到记录检查，再切换到编码与发布，同时将观测压缩为当前阶段所需的模式。由此，模型创建了一台阶段机，因为任务的模态会从证据获取转变为产物构建。

而 PEGBOARD 处理的是一个由六条人物生平与电影线索定义的身份识别问题。它将进度表示为候选对象 × 线索矩阵，其中每条提取出的主张都保留其文档标识符。空单元格和矛盾会驱动

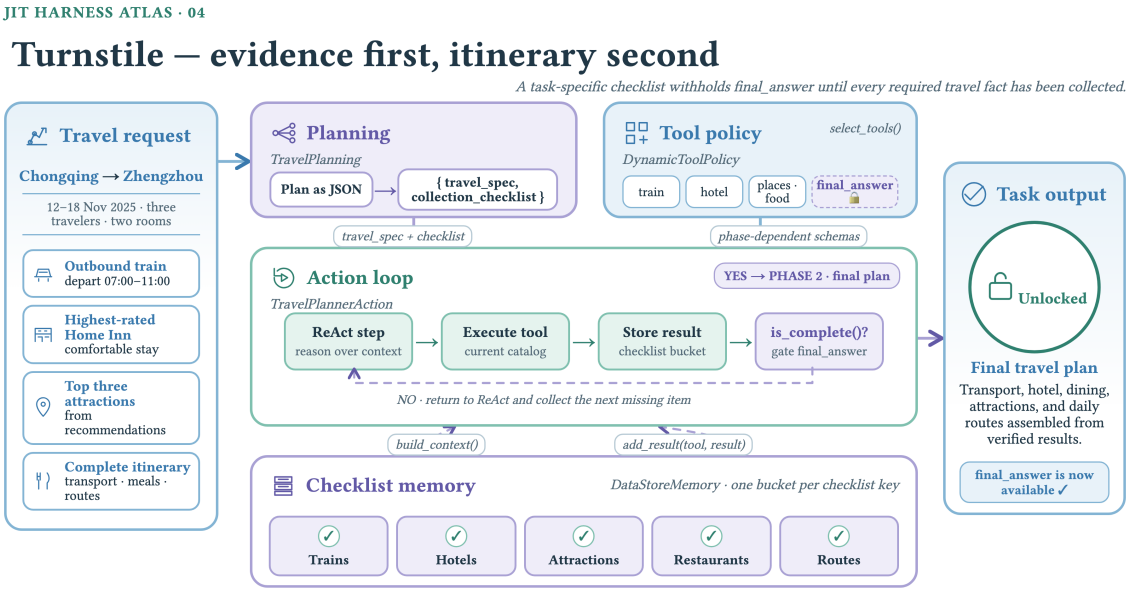


图 **A.2: Turnstile**：证据优先，行程在后。`TravelPlanning` 生成旅行规格与检查清单，`DataStoreMemory` 跟踪所需的证据桶，`DynamicToolPolicy` 仅在 `is_complete()` 成功后开放 `final_answer`。

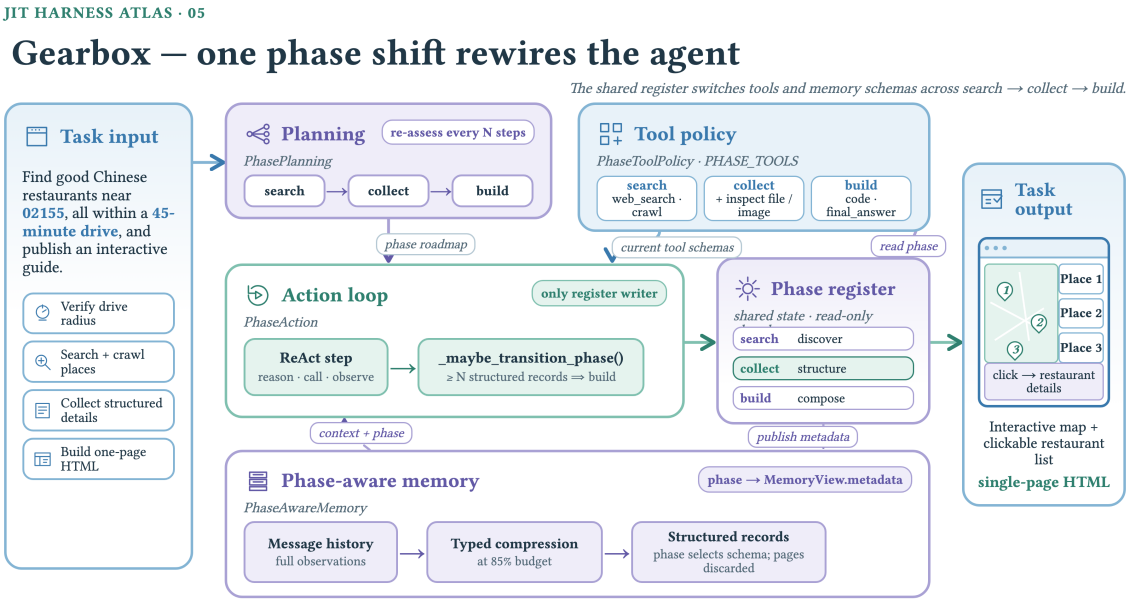


图 **A.3: Gearbox**：一次阶段切换即可重写智能体。`PhaseAction` 是共享阶段寄存器的唯一写入者；`PhaseToolPolicy` 和 `PhaseAwareMemory` 读取该状态，以同时切换开放能力和类型化记忆模式。

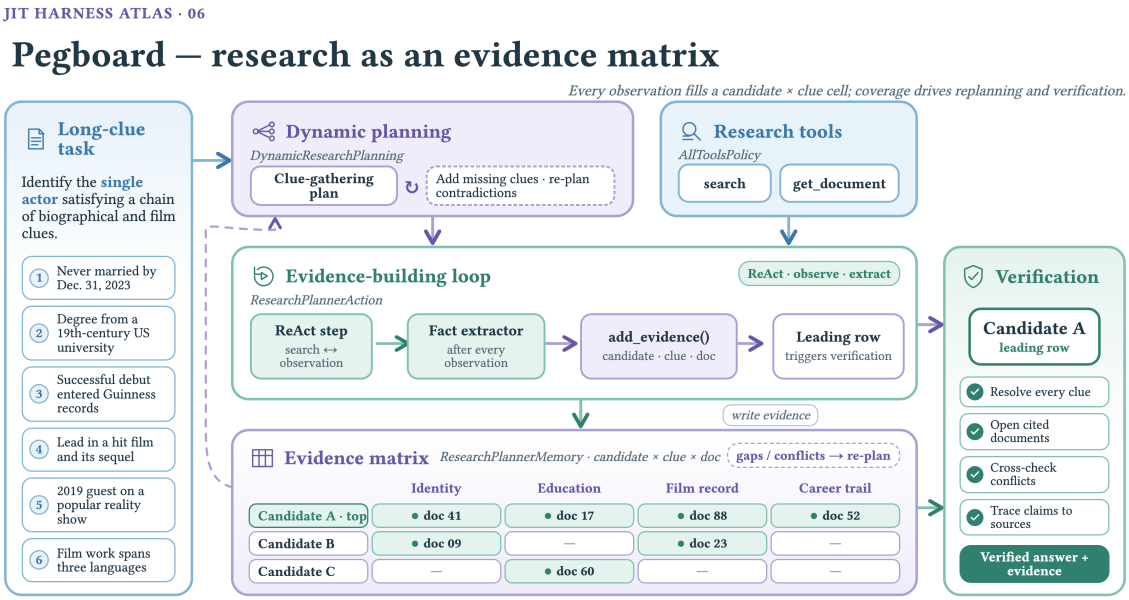


图 **A.4: Pegboard**：将研究表示为证据矩阵。每条观测都被提取到一个带文档标识符的候选对象 × 线索单元格中；矩阵覆盖率同时驱动 `DynamicResearchPlanning` 以及向来源有据的验证阶段转移。

`DynamicResearchPlanning` 进行定向搜索，而得到充分支持的一行会触发独立的验证阶段，重新打开所引文档并解决冲突。与通用研究记录不同，这一状态使覆盖度与来源溯源可以直接驱动动作。它与 GEARBOX 的对比表明，生成状态既可以是一个很小的阶段变量，也可以是任务塑造的证据结构，具体取决于下一步动作需要由什么来控制。

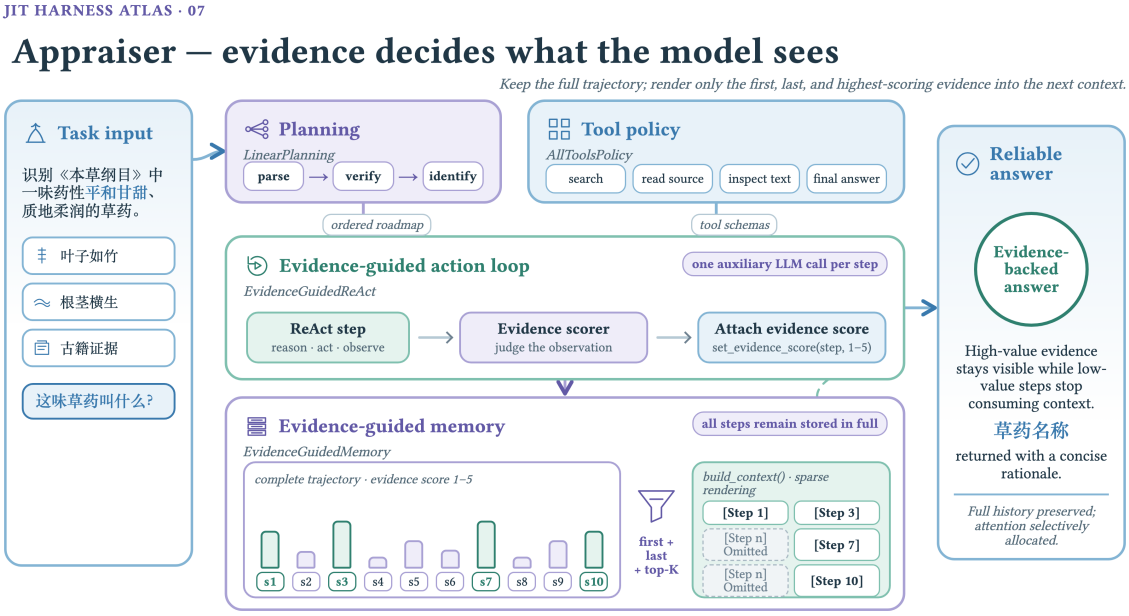


图 **A.5: Appraiser**：由证据决定模型看到什么。`EvidenceGuidedReAct` 为每一步评分，而 `EvidenceGuidedMemory` 存储完整运行过程，但仅将第一条、最后一条和得分前 K 的观测渲染进 `build_context()`。

图 A.5 和 A.6 对后续推理从先前执行中接收的内容进行定制。APPRAISER 通过协调植物学特征与文

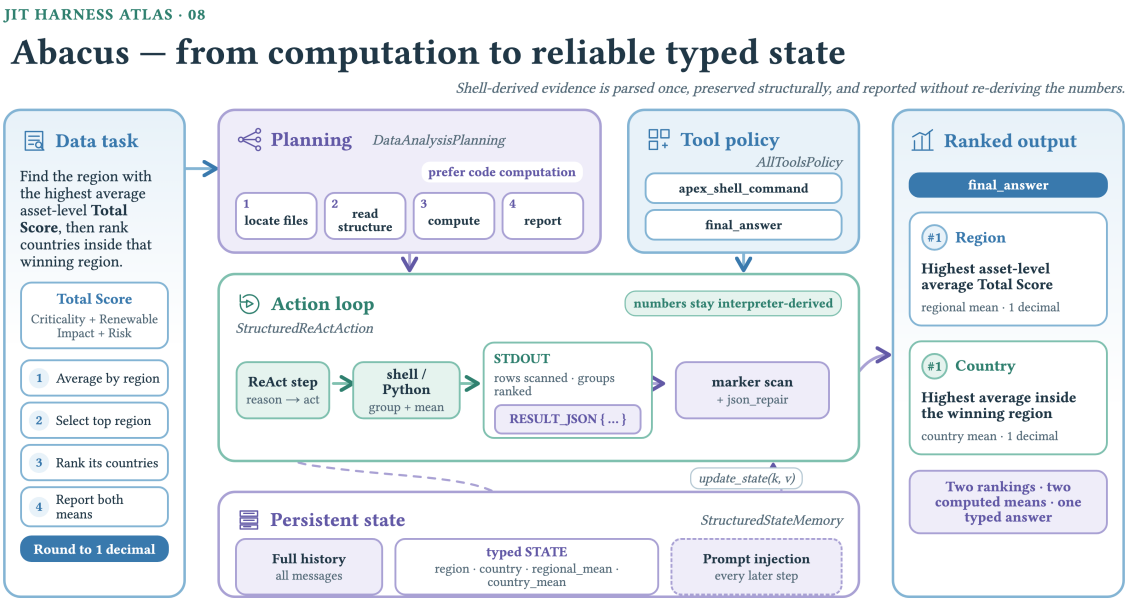


图 **A.6: Abacus**：从计算到可靠的类型化状态。`StructuredReActAction` 执行 shell 或 Python 代码，从标准输出中提取 `RESULT_JSON` 载荷，并更新 `StructuredStateMemory`，无需让模型重新推导计算值。

本证据来回答一个传统本草鉴定问题。每次搜索或阅读后，辅助评分器都会为该观测赋予证据价值；`EvidenceGuidedMemory` 保留完整轨迹以供审计，但仅将第一条、最后一条和得分前 K 的观测渲染到下一轮上下文中。于是，低价值探索不再与最有助于鉴定的段落争夺上下文，同时没有任何证据从持久历史中被删除。该支架专为一种主要瓶颈在于证据显著性而非工具选择的检索任务定制。

Abacus 是为一个数值分析请求生成的，该请求先比较区域平均值，再对胜出区域内的国家进行排名。支架刻意将聚合移出语言模型推理：`StructuredReActAction` 调用 shell 或 Python，从标准输出中提取带标记的 `RESULT_JSON` 载荷，必要时修复其语法，并将数值写入 `StructuredStateMemory`。随后，区域、国家、区域均值和国家均值以类型化状态注入之后的每一步，因此最终答案报告由解释器计算得出的量，而不是从文字中重新推导。两个案例都保留完整审计轨迹，但各自围绕其任务最主要的可靠性要求渲染工作视图：前者是证据选择，后者是数值保真。

图 A.7 和 A.8 通过控制动作何时进入持久历史，针对不同规模的工作空间执行进行特化。Player Piano 处理配置文件的批量迁移，包括数据库、日志记录和 API 端点变更。`StructuredPlanning` 生成类型化的发现、处理文件和最终答案步骤，并由 Python 调度器执行；每次编辑后，一个确定性的非 LLM 检查决定继续还是重试。`ProgressiveTaskMemory` 将逐文件看板渲染到系统提示中，同时保留原始内容和已接受的更改，防止大批量任务遗忘哪些文件已经真正通过验证。

Mulligan 是为一个较短的文档任务生成的：从 DOCX 文件中提取学生标识符，并重命名同一个文件。其线性计划无需层级结构或上下文压缩，因此支架保留完整的成功历史，仅开放 Word 和文件操作，同时隐藏无关的电子邮件、电子表格和 PDF 工具。如果动作失败，循环会在局部追加错误，并在同一步骤重新生成动作，最多尝试两次；只有成功动作会在计划推进前提交。因此，这两个支架都关注可靠的工作空间修改，但实现方式不同：一个使用类型化批量进度和逐文件验证，另一个则使用狭窄的工具开放范围和事务式步骤修复。这种依任务而异的差异，正是 JIT-Agent 所追求的行为：可靠性机制根据请求的结构和故障面生成，而不是以一种通用脚手架强加给所有任务。

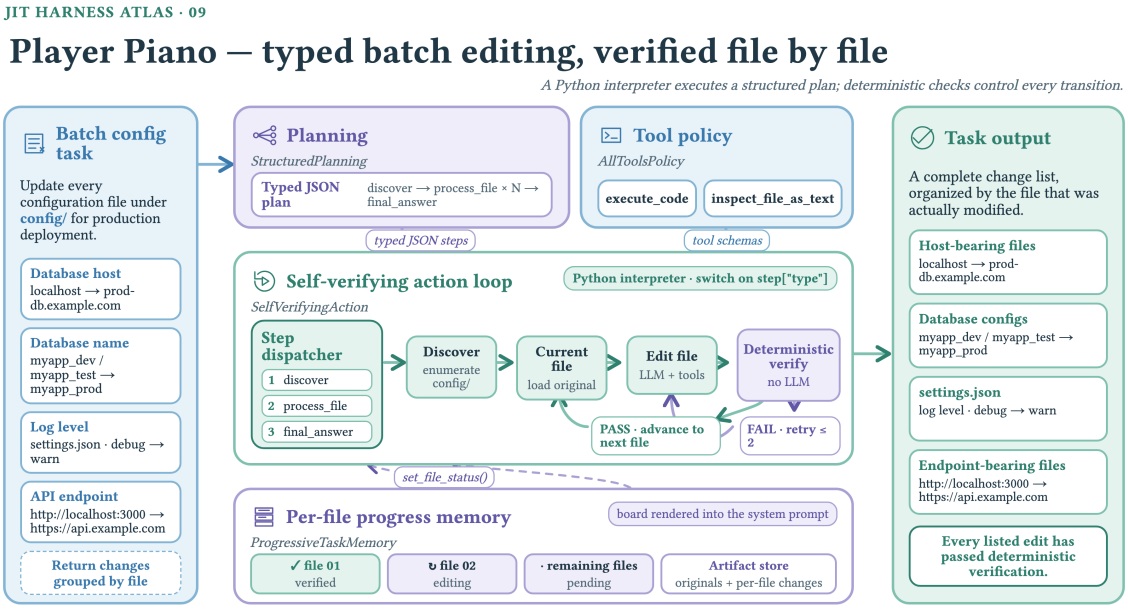


图 **A.7: Player Piano**：类型化批量编辑，逐文件验证。Python 调度器解释 `StructuredPlanning` 步骤，`SelfVerifyingAction` 在不使用 LLM 的情况下检查每次编辑，`ProgressiveTaskMemory` 则维护逐文件看板和产物存储。

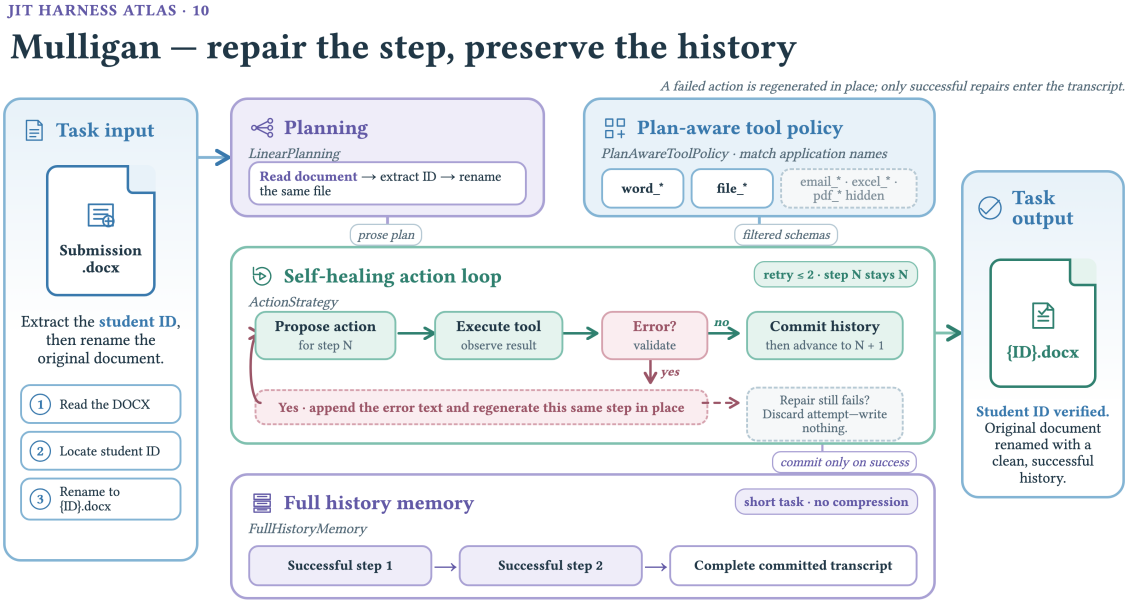


图 **A.8: Mulligan**：修复当前步骤，保留既有历史。`PlanAwareToolPolicy` 仅开放与计划相关的文件和文档工具；失败动作会在同一步骤重新生成，而 `FullHistoryMemory` 只接收成功执行。